

# Computer Graphics (CSIT 401)

III semester, BScCSIT

Compiled by **Ankit Bhattarai**  
Ambition Guru College

# Syllabus

| Unit      | Contents                                       | Hours    | Remarks                      |
|-----------|------------------------------------------------|----------|------------------------------|
| 1.        | Introduction to Computer Graphics              | 5        |                              |
| 2.        | Scan Conversion Algorithms                     | 7        | <i>Very Important</i>        |
| <b>3.</b> | <b>2D Transformations and Viewing</b>          | <b>6</b> | <b><i>Very Important</i></b> |
| 4.        | 3D Transformations and Viewing                 | 5        |                              |
| 5.        | 3D Object Representation                       | 6        |                              |
| 6.        | Solid Modeling                                 | 3        |                              |
| 7.        | Visible Surface Detection                      | 5        |                              |
| 8.        | Illumination, Shading and Rendering Techniques | 5        |                              |
| 9.        | OpenGL and Graphics Programming                | 3        |                              |

## Unit 3

# 2D Transformation and Viewing

(6 hrs.)

Coordinate systems, Basic 2D transformations: Translation, rotation, scaling, reflection, shear, Homogeneous coordinates and matrix representation, Composite transformations, 2D viewing pipeline, Window-to-viewport coordinate transformation.

Clipping: Introduction, clipping Window, Normalization and viewport transformations, Point Lines, Point and Lines Clipping Algorithm (Cohen-Sutherland, Liang-Barsky), Polygon fill area clipping: Sutherland-Hodgeman Polygon Clipping Algorithm.

## Section 1: 2D Geometric Transformation

# Introduction

## Basic Transformations

- Transformations means changing the graphics by changing the position, orientation or size of the original graphics by applying rules.
- When transformations occur in 2D plane then it is called 2D transformations.
- The orientation, size, and shape of the output primitives are accomplished with geometric transformations that alter the coordinate descriptions of objects.
- The basic geometric transformations are **translation**, **rotation**, and **scaling**. It also includes **reflection** and **shearing**.

## HOMOGENOUS FORM

Expressing position in homogeneous coordinates allows us to represent all geometric transformations equation as matrix multiplications.

$$(x, y) \text{ ----- } (x_h, y_h, h)$$

$$x = x_h / h \quad y = y_h / h$$

where h is any non zero value

For convenient  $h=1$

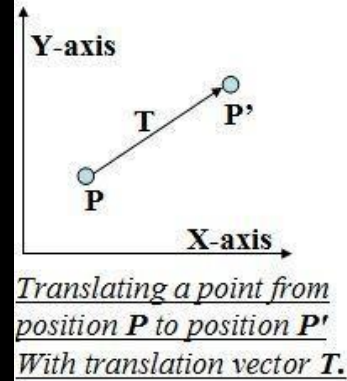
$$(x, y) \text{ ----- } (x, y, 1).$$

# 1. Translation

A translation is applied to an object by repositioning it along a straight-line path from one coordinate location to another. We translate a two-dimensional point by adding translation distances,  $t_x$ , and  $t_y$ , to the original coordinate position  $(x, y)$  to move the point to a new position  $(x', y')$ .

$$x' = x + t_x$$

$$y' = y + t_y \text{ where the pair } (t_x, t_y) \text{ is called the } \textbf{translation vector} \text{ or } \textbf{shift vector}.$$



# 1. Translation

We can write equation as a single matrix equation by using column vectors to represent coordinate points and translation vectors. Thus,

$$P = \begin{bmatrix} x \\ y \end{bmatrix} \quad T = \begin{bmatrix} t_x \\ t_y \end{bmatrix} \quad P' = \begin{bmatrix} x' \\ y' \end{bmatrix}$$

$$P' = P + T$$

So we can write

In homogeneous representation if position  $P = (x, y)$  is translated to new position  $P' = (x', y')$  then:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \Rightarrow P' = T(t_x, t_y).P$$



**Question:**

Translate the given points  $(2,5)$  by the translating value  $(3,3)$ .

Solution:

New coordinates of given point =  $(5, 8)$

**Question:**

Translate the given square having coordinate A(0,0), B(3,0), C(3,3) and D(0,3) by the translating value 2 in both directions.

Solution:

New coordinates are A'(2, 2), B'(5,2), C'(5,5) and D'(2,5)

# Exercises

**Q. Translate the given square having coordinate A (0,0), B (3,0), and C (3,3) and D ( 0, 3) by the translating value 2 in both directions.**

Solution:

Given points A (0,0), B (3,0), and C (3,3) and D ( 0, 3) and translating value 2 ( $t_x=t_y=2$ )

We have From Translation Matrix

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

i.e.

$$A' = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 2 \\ 2 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

$$B' = \begin{bmatrix} 3 \\ 0 \end{bmatrix} + \begin{bmatrix} 2 \\ 2 \end{bmatrix} = \begin{bmatrix} 5 \\ 2 \end{bmatrix}$$

$$C' = \begin{bmatrix} 3 \\ 3 \end{bmatrix} + \begin{bmatrix} 2 \\ 2 \end{bmatrix} = \begin{bmatrix} 5 \\ 5 \end{bmatrix}$$

$$D' = \begin{bmatrix} 0 \\ 3 \end{bmatrix} + \begin{bmatrix} 2 \\ 2 \end{bmatrix} = \begin{bmatrix} 2 \\ 5 \end{bmatrix}$$

Hence the final co-ordinate are A' (2,2), B'(5,2), C'(5,5) and D'(2,5)

**Question:**

**Given a circle C with radius 10 and center coordinates (1, 4). Apply the translation with distance 5 towards X axis and 1 towards Y axis. Obtain the new coordinates of C without changing its radius**

Solution:

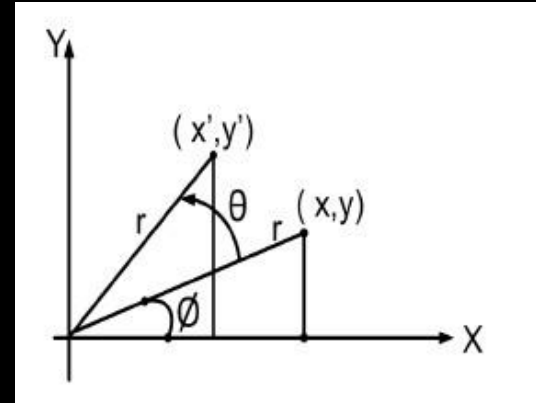
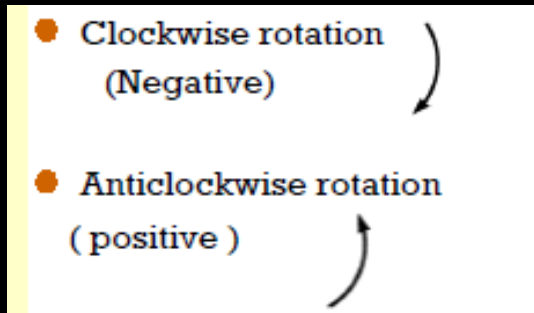
New center coordinates of C = (6, 5)

## 2. ROTATION

A two-dimensional rotation is applied to an object by repositioning it along a circular path in the **xy** plane. To generate a rotation, we specify a rotation angle  $\theta$  and the position  $(x_r, y_r)$  of the rotation point (or pivot point) about which the object is to be rotated.

+ Value for ' $\theta$ ' define *counter-clockwise* rotation about a point

- Value for ' $\theta$ ' defines *clockwise* rotation about a point



## 2. ROTATION

Let  $(x, y)$  is the original point, ' $r$ ' the constant distance from origin, & ' $\Phi$ ' the original angular displacement from x-axis. Now the point  $(x, y)$  is rotated through angle ' $\theta$ ' in a counter clock wise direction

we can express the transformed coordinates in terms of ' $\Phi$ ' and ' $\theta$ ' as

$$x' = r \cos(\Phi + \theta) = r \cos\Phi \cdot \cos\theta - r \sin\Phi \cdot \sin\theta \dots(i)$$

$$y' = r \sin(\Phi + \theta) = r \cos\Phi \cdot \sin\theta + r \sin\Phi \cdot \cos\theta \dots(ii)$$

We know that original coordinates of point in polar coordinates are

$$x = r \cos \Phi$$

$$y = r \sin \Phi$$

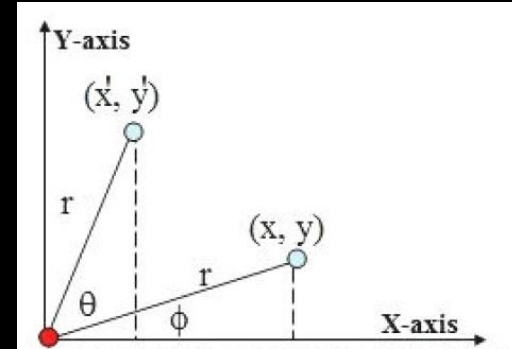
Substituting these values in (i) and (ii), we get,

$$x' = x \cos\theta - y \sin\theta$$

$$y' = x \sin\theta + y \cos\theta$$

So using column vector representation for coordinate points the matrix form would be  $P' = R \cdot P$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$



Anticlockwise direction

## 2. ROTATION

In homogeneous co-ordinate

Anticlockwise direction

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \Rightarrow P' = R(\theta).P$$

Clockwise direction

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \Rightarrow P' = R(\theta).P$$

## 2. ROTATION: Rotation of a point about an arbitrary pivot position

- Translate the point  $(x_r, y_r)$  and  $P(x, y)$  by translation vector  $(-x_r, -y_r)$  which translates the pivot to origin and  $P(x, y)$  to  $(x - x_r, y - y_r)$ .
- Now apply the rotation equations when pivot is at origin to rotate the translated point  $(x - x_r, y - y_r)$  as:

$$x_1 = (x - x_r)\cos\theta - (y - y_r)\sin\theta$$

$$y_1 = (x - x_r)\sin\theta + (y - y_r)\cos\theta$$

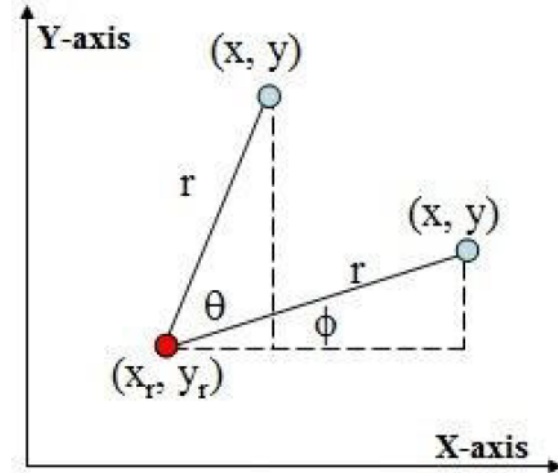
- Re- translate the rotated point  $(x_1, y_1)$  with translation vector  $(x_r, y_r)$  which is reverse translation to original translation. Finally we get the equation after successive transformation as

$$x' = x_r + (x - x_r)\cos\theta - (y - y_r)\sin\theta \dots\dots\dots 1$$

$$y' = y_r + (x - x_r)\sin\theta + (y - y_r)\cos\theta \dots\dots\dots 2$$

Which are actually the equations for rotation of  $(x, y)$  from the pivot point  $(x_r, y_r)$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} 1-\cos\theta & -\sin\theta \\ -\sin\theta & 1-\cos\theta \end{bmatrix} \begin{bmatrix} x_r \\ y_r \end{bmatrix}$$





**Question:**

Given a line segment with starting point as  $(0, 0)$  and ending point as  $(4, 4)$ . Apply 30 degree rotation anticlockwise direction on the line segment and find out the new coordinates of the line.

Solution:

New ending coordinates of the line after rotation =  
 $(1.46, 5.46)$

**Question:**

Given a triangle with corner coordinates A(0, 0), B(1, 0) and C(1, 1). Rotate the triangle by 90 degree anticlockwise direction and find out the new coordinates.

**Solution:**

New coordinates of corner A after rotation = (0, 0)

New coordinates of corner B after rotation = (0, 1)

New coordinates of corner C after rotation = (-1, 1)



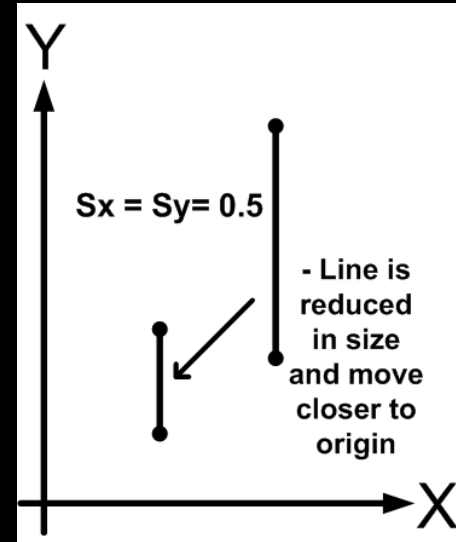
### 3. Scaling

- A scaling transformation alters the size of an object. This operation can be carried out for polygons by multiplying the coordinate values  $(x, y)$  of each vertex by scaling factors  $S_x$  and  $S_y$  to produce the transformed coordinates  $(x', y')$ .
- $S_x$  scales object in 'x' direction
- $S_y$  scales object in 'y' direction

Thus, for equation form,

$$x' = x \cdot S_x \text{ and}$$

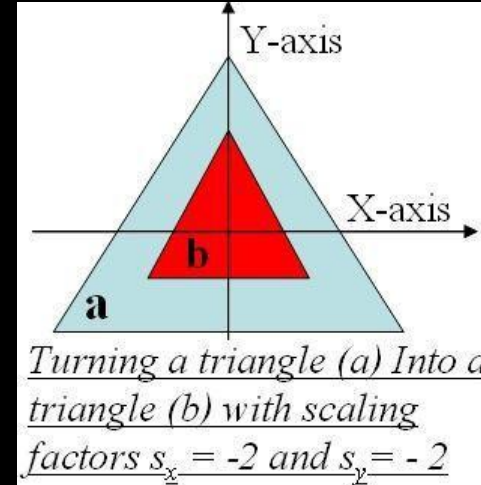
$$y' = y \cdot S_y$$



### 3. Scaling

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

- Values greater than 1 for  $s_x$  ,  $s_y$  produce **enlargement**
- Values less than 1 for  $s_x$  ,  $s_y$  **reduce** size of object
- $s_x = s_y = 1$  leaves the size of the object **unchanged**
- When  $s_x$  ,  $s_y$  are assigned the same value  $s_x = s_y = 3$  or 4 etc then a **Uniform Scaling** is produced



**In homogeneous co-ordinate**

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \Rightarrow P' = S(s_x, s_y).P$$

$$P' = S.P$$

### 3. Scaling: Scaling About arbitrary point

If  $p(x,y)$  be scaled to a point  $p'(x', y')$  by  $s_x$  time in X-units and  $s_y$  times in y-units about arbitrary point  $(x_f, y_f)$  then the equation for scaling is given as

$$X' = X_f + S_x \cdot (X - X_f)$$

$$y' = y_f + S_y \cdot (y - y_f)$$

**NOTE: Arbitrary axis** simply means the axis chosen in any way we like within the co-ordinate system. We choose any line in space, and then put a pin into our object along that line, and transform the object around the pin.

**Question:**

Given a square object with coordinate points A(0, 3), B(3, 3), C(3, 0), D(0, 0).

Apply the scaling parameter 2 towards X axis and 3 towards Y axis and obtain the new coordinates of the object.

Solution:

New coordinates of corner A after scaling = (0, 9)

New coordinates of corner B after scaling = (6, 9)

New coordinates of corner C after scaling = (6, 0)

New coordinates of corner D after scaling = (0, 0).

## 4. Reflection

- A reflection is a transformation that produces a mirror image of an object. The mirror image for a 2D reflection is generated relative to an axis of reflection by rotating the object  $180^\circ$  about the reflection axis.
- We can choose an axis of reflection in the **xy**-plane or perpendicular to the **xy** plane. When the reflection axis is a line in the **xy** plane, the rotation path about this axis is in a plane perpendicular to the **xy**-plane.
- For reflection axes that are perpendicular to the **xy**-plane, the rotation path is in the **xy** plane.

## 4. Reflection

### (i) Reflection about x axis or about line $y = 0$

Keeps **X** value same but flips **Y** value of coordinate points

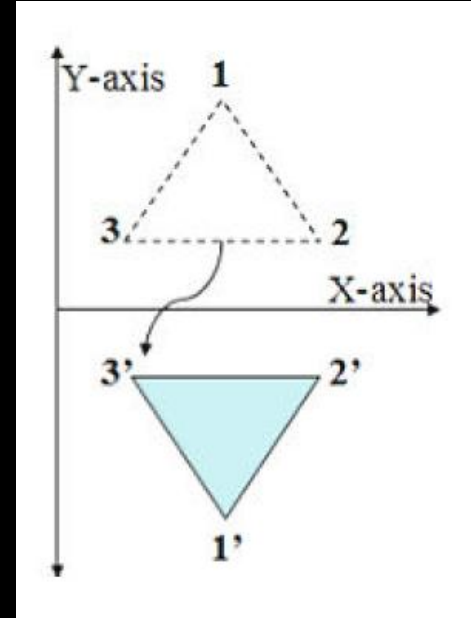
$$x' = x$$

$$y' = -y$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Homogeneous co-ordinate

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$





## 4. Reflection

### (ii) Reflection about y axis or about line $x = 0$

- Keeps 'y' value same but flips x value of
- coordinate
- points
- $x' = -x$
- $y' = y$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Homogeneous co-ordinate

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

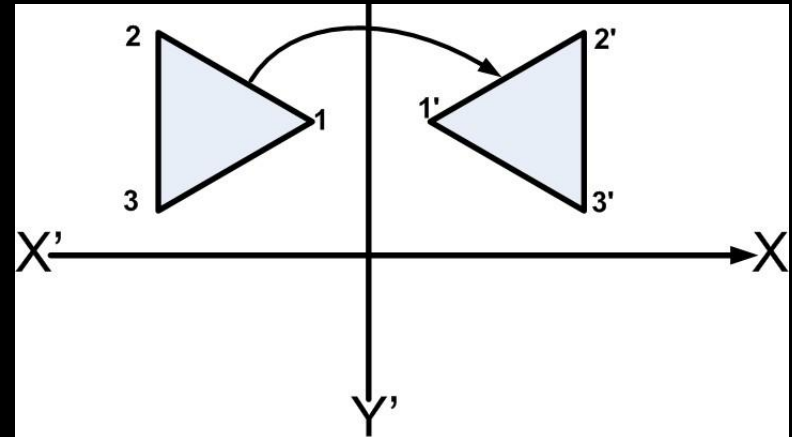


Fig: Reflection of object about y-axis ( $x=0$ )

## 4. Reflection



AMBITION GURU

### (iii) Reflection about origin

Flip both 'x' and 'y' coordinates of a point

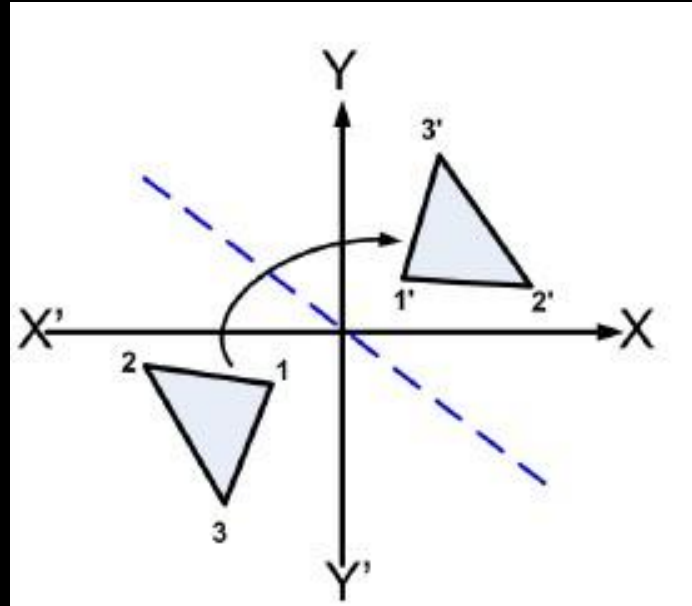
$$x' = -x$$

$$y' = -y$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Homogeneous co-ordinate

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



## 4. Reflection

### (iv) Reflection about line $y = x$

$$x' = y$$

$$y' = x$$

Thus, reflection against  
 $x=y$ -axis (i.e.  $\theta = 45^\circ$ )

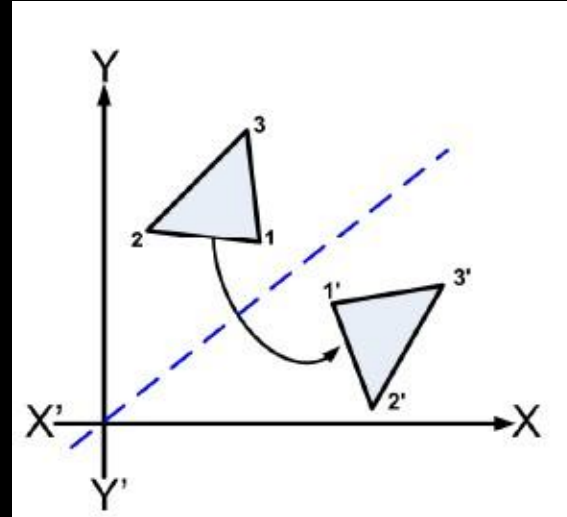
Hence

$$R_{x=y} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Homogeneous co-ordinate

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



Equivalent to:

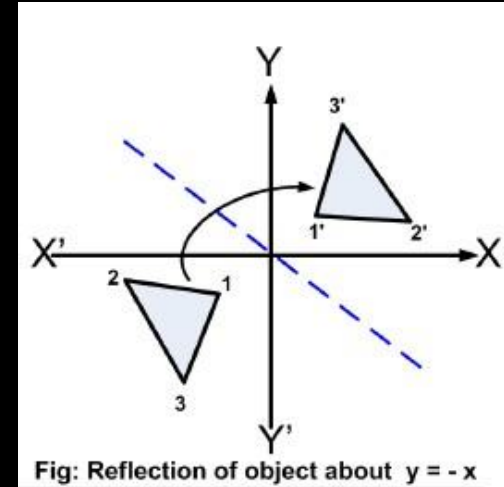
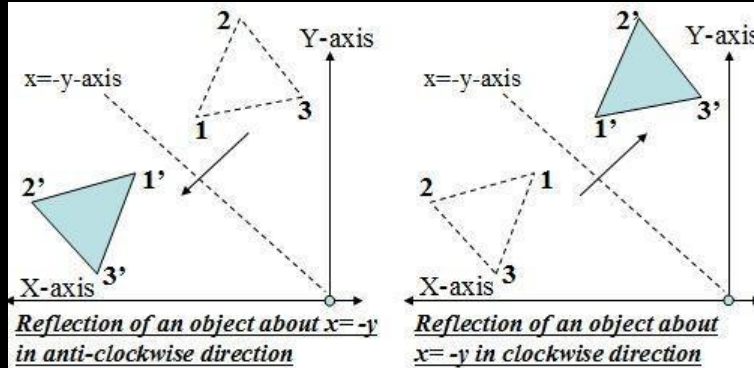
- Reflection about x-axis
- Rotate anticlockwise  $90^\circ$   
**OR**
- Clockwise rotation  $45^\circ$
- Reflection with x-axis
- anticlockwise rotation  $45^\circ$

# 4. Reflection

## (v) Reflection about line $y = -x$

$$x' = -y$$

$$y' = -x$$



Thus, reflection against  $x=y$ -axis  
in anti-clockwise direction (i.e.  $\theta = 45$ )

**Hence**

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \text{ Where } R_{x=y} = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix}$$

Thus, reflection against  $x=y$ -axis  
in clockwise direction (i.e.  $\theta = -45$ )

**Hence**

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \text{ Where } R_{x=y} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

**Question:**

Given a triangle with coordinate points A(3, 4), B(6, 4), C(5, 6). Apply the reflection on the X axis and obtain the new coordinates of the object.

**Solution:**

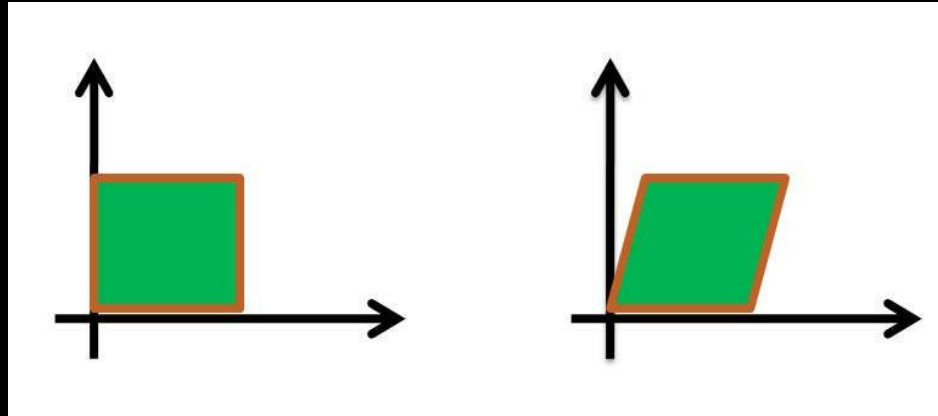
New coordinates of corner A after reflection = (3, -4)

New coordinates of corner B after reflection = (6, -4)

New coordinates of corner C after reflection = (5, -6)

## 5. Shearing

- It distorts the shape of object in either 'x' or 'y' or both direction. In case of single directional shearing (e.g. in 'x' direction can be viewed as an object made up of very thin layer and slid over each other with the *base* remaining where it is).
- Shearing is a ***non-rigid-body transformation*** that moves objects with deformation.



# 5. Shearing

## X-direction Shear:

An X-direction shear relative to x-axis is produced with transformation matrix equation.

In 'x' direction,

$$x' = x + S_{hx} \cdot y$$

$$y' = y$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & S_{hx} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

## Y-direction Shear:

An y-direction shear relative to y-axis is produced by following transformation equations.

In 'y' direction,

$$x' = x$$

$$y' = y + S_{hy} \cdot x$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ S_{hy} & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

## Both direction Shear:

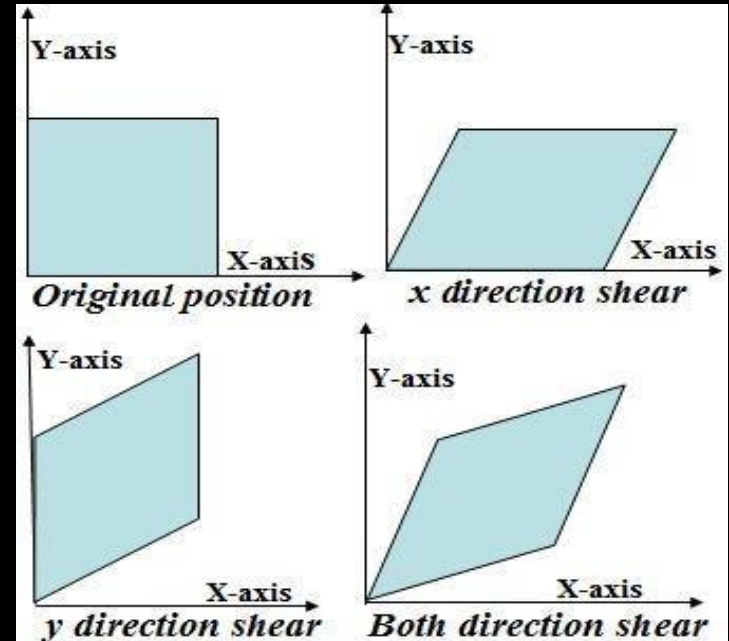
Both directions share relative to x-axis and y-axis is produced by following transformation equations

In both directions,

$$x' = x + S_{hx} \cdot y$$

$$y' = y + S_{hy} \cdot x$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & S_{hx} \\ S_{hy} & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$



**Fig. Two Dimensional shearing**

**Question:**

**Given a triangle with points A(1, 1), B(0, 0) and C(1, 0). Apply shear parameter 2 on X axis and 2 on Y axis and find out the new coordinates of the object.**

Solution:

**Shearing in X Axis-**

New coordinates of corner A after shearing = (3, 1)

New coordinates of corner B after shearing = (0, 0)

New coordinates of corner C after shearing = (1, 0)

**Shearing in Y Axis-**

New coordinates of corner A after shearing = (1, 3)

New coordinates of corner B after shearing = (0, 0)

New coordinates of corner C after shearing = (1, 2)



## Section 2: Homogenous Coordinates, Composite Transformation

# Homogenous Coordinates

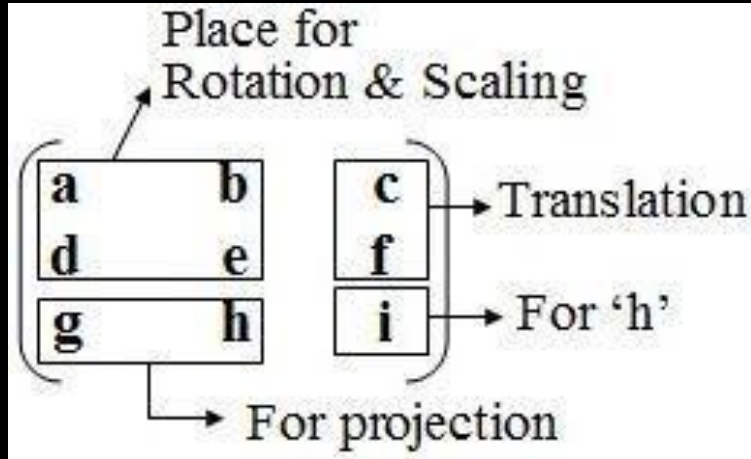
The matrix representations for translation, scaling and rotation are respectively:

- Translation:  $\mathbf{P}' = \mathbf{T} + \mathbf{P}$  (*Addition*)
- Scaling:  $\mathbf{P}' = \mathbf{S} \cdot \mathbf{P}$  (*Multiplication*)
- Rotation:  $\mathbf{P}' = \mathbf{R} \cdot \mathbf{P}$  (*Multiplication*)

Since, the composite transformation such as include many sequence of translation, rotation etc and hence the many naturally differ addition & multiplication sequence have to perform by the graphics allocation. Hence, the applications will take more time for rendering.

Thus, we need to treat all three transformations in a consistent way so they can be combined easily & compute with one mathematical operation.

# Homogenous Coordinates



- Here, in case of homogenous coordinates we add a third coordinate ' $h$ ' to a point  $(x, y)$  so that each point is represented by  $(hx, hy, h)$ .
- The ' $h$ ' is normally set to 1. If the value of ' $h$ ' is more than one value then all the co-ordinate values are scaled by this value.

# Homogenous Coordinates

Coordinates of a point are represented as three element column vectors; transformation operations are written as 3 x 3 matrices.

**For translation**

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

With T ( $t_x$ ,  $t_y$ ) as translation matrix, inverse of this translation matrix is obtained by representing  $t_x$ ,  $t_y$  with  $-t_x$ ,  $-t_y$ .

**For rotation**

a. Counter Clock Wise (CCW):

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

b. Clock Wise (CCW):

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

- **For scaling**

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} S_{hx} & 0 & 0 \\ 0 & S_{hy} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

- **For Reflection**

- Reflection about x-axis

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

- Reflection about y-axis

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

- Reflection about y=x-axis

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

- Reflection about y=-x-axis

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 0 & -1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

- Reflection about any line y=mx+c

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{-(m^2-1)}{(m^2+1)} & \frac{2m}{(m^2+1)} & \frac{2mc}{(m^2+1)} \\ \frac{2m}{(m^2+1)} & \frac{(m^2-1)}{(m^2+1)} & \frac{2c}{(m^2+1)} \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

# Composite Transformation

- With the matrix representation of transformation equations it is possible to setup a matrix for any sequence of transformations as a composite transformation matrix by calculating the matrix product of individual transformation. Forming products of transformation matrices is often referred to as a **concatenation**, or **composition**, of matrices. For column matrix representation of coordinate positions we form composite transformation by multiplying matrices in order from right to left.

**Right to Left**

$$\mathbf{A.B \neq B.A}$$

# Composite Transformation

- When more than one transformations are applied for performing a task then such transformation is called **composite transformation**.
- Basic purpose of composing transformations is to gain efficiency by applying a single composed transformation to a point, rather than applying a series of transformation, one after another.

To perform a sequence of transformation such as translation followed by rotation and scaling, we need to follow a sequential process:

- Translate the coordinates,
  - Rotate the translated coordinates, and then
  - Scale the rotated coordinates to complete the composite transformation.
- ✓ To shorten this process, we have to use  $3 \times 3$  transformation matrix instead of  $2 \times 2$  transformation matrix. To convert a  $2 \times 2$  matrix to  $3 \times 3$  matrix, we have to add an extra dummy coordinate. So, we can represent the point by 3 numbers instead of 2 numbers, which is called **Homogenous Coordinate system**.

In this system, we can represent all the transformation equations in matrix multiplication.

NOTE: for numerical **ALWAYS REMEMBER !!!**

In composite transformation, we *multiply the individual transformation matrices in the reverse order* of how they're applied to get the combined transformation matrix.

Let's say you have matrices A, B, and C representing transformations. The composite transformation matrix (M) would be  $\mathbf{M} = \mathbf{C} * \mathbf{B} * \mathbf{A}$ .

Representing a triangle for coordinates (a,b), (c,d), (e,f) in homogeneous form would be

$$\begin{bmatrix} a & c & e \\ b & d & f \\ 1 & 1 & 1 \end{bmatrix}$$

**Note:**  $P' = R \cdot P$

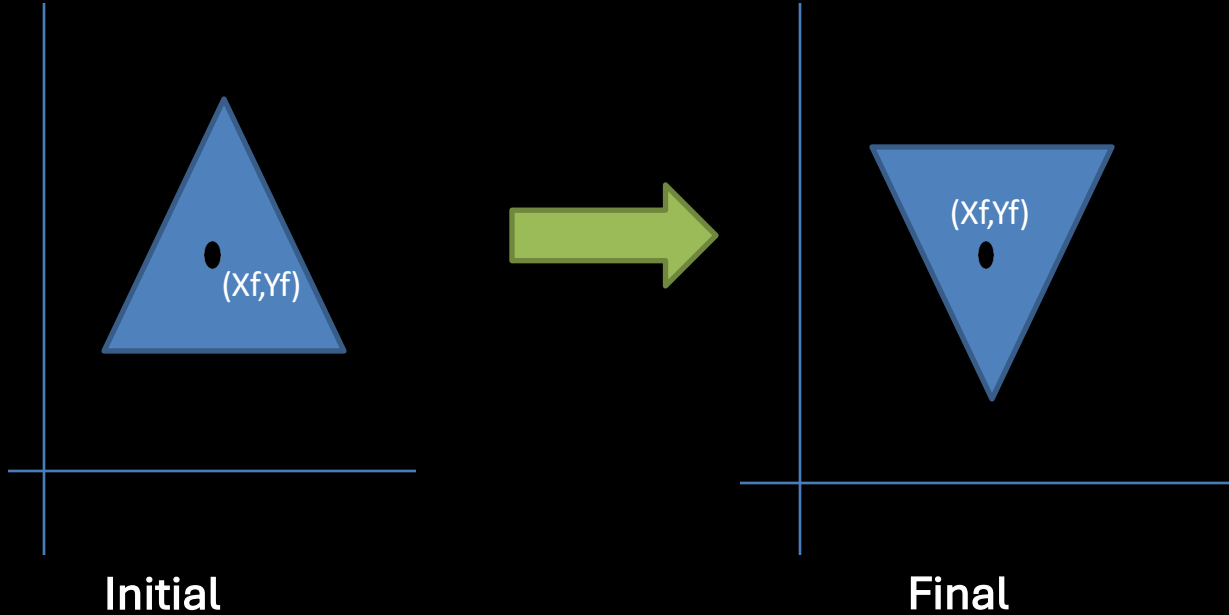


$R$  can be any transformations.



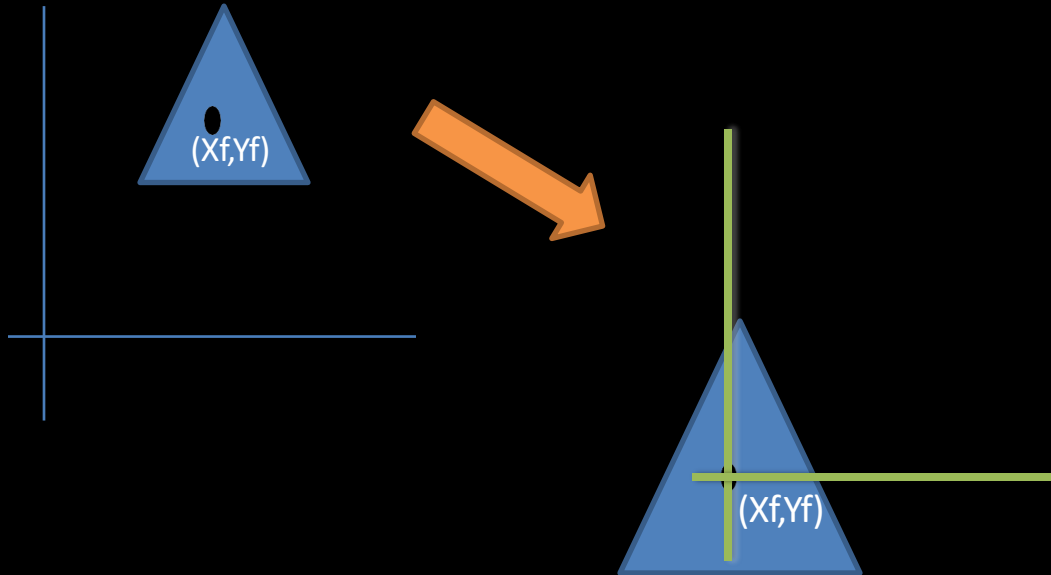
# CASE: Fixed Point Rotation

# CASE: Fixed Point Rotation



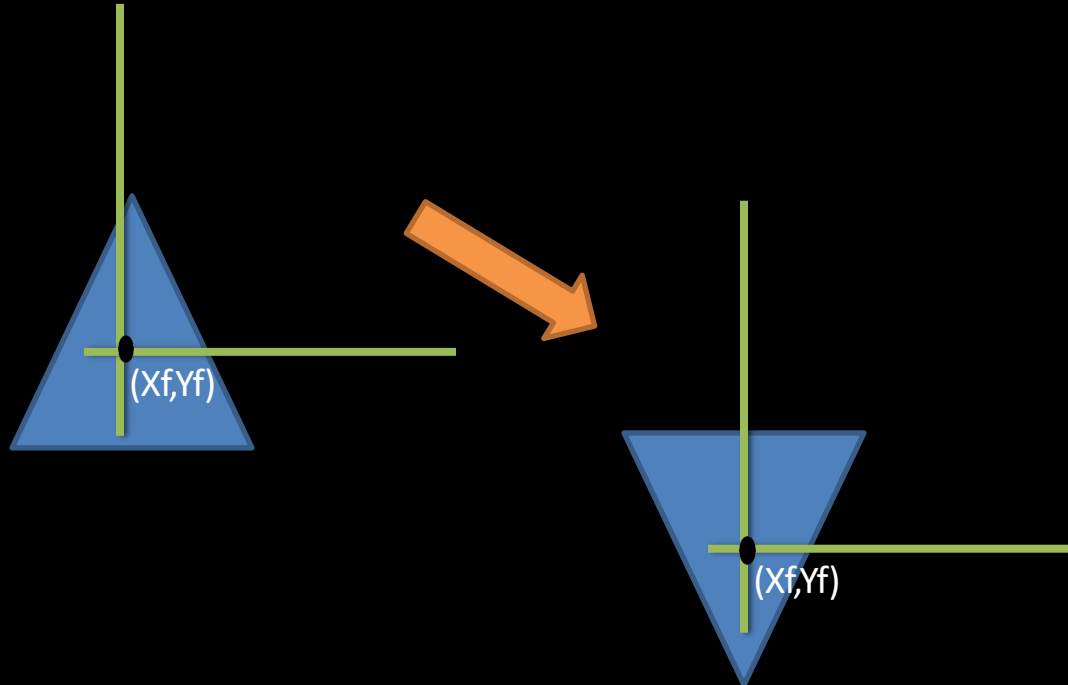
# Fixed Point Rotation

Step 1: The fixed point along with the object is translated to coordinate origin.



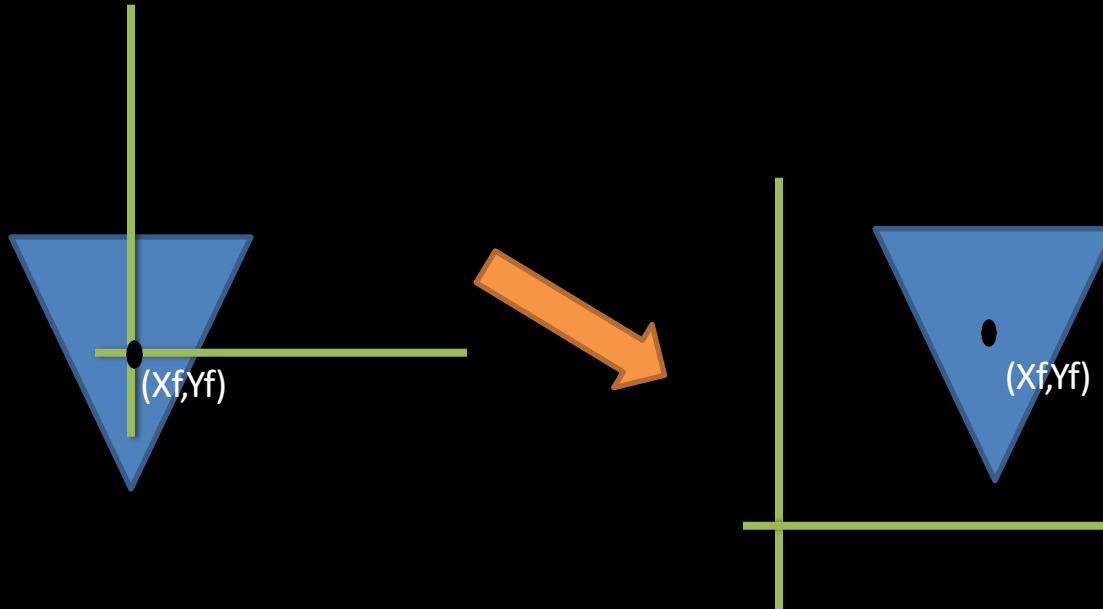
# Fixed Point Rotation

Step 2: Rotate the object about origin

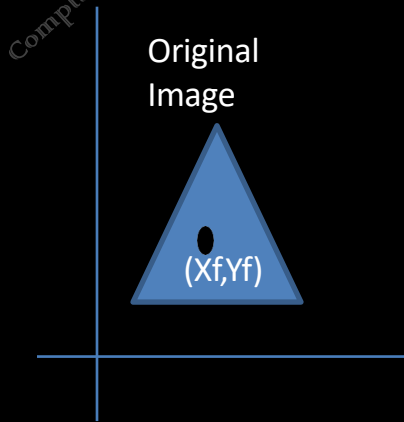


# Fixed Point Rotation

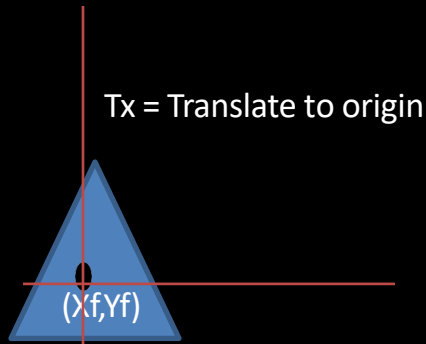
Step 3: The fixed point along with the object is translated back to its original position.



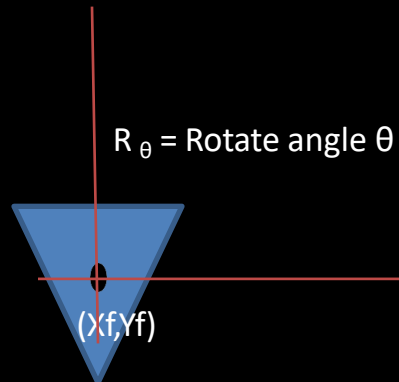
# Fixed Point Rotation



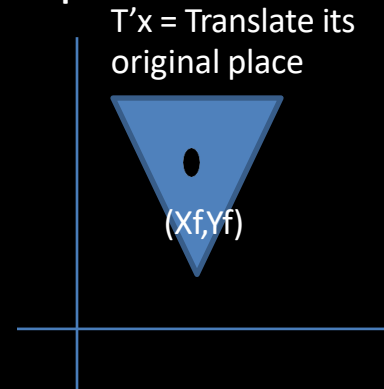
**Step 1**



**Step 2**



**Step 3**



# Fixed Point Rotation

Composite Transformation

$$= T'_{(x_f, y_f)} \cdot R_{\theta} \cdot T_{(-x_f, -y_f)}$$

$$= \begin{bmatrix} 1 & 0 & x_f \\ 0 & 1 & y_f \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -x_f \\ 0 & 1 & -y_f \\ 0 & 0 & 1 \end{bmatrix}$$

$T'_x$  = Translate back to fixed point

$R_{\theta}$  = Rotate angle  $\theta$

$T_x$  = Translate fixed point to origin



## Exercise 1

**Q. N.:** Rotate the triangle (5, 5), (7, 3), (3, 3) in counter clockwise (CCW) by 90 degree.

**Solution:**

$$P' = R \cdot P$$

$$\begin{aligned}
 &= \begin{pmatrix} \cos 90^\circ & -\sin 90^\circ & 0 \\ \sin 90^\circ & \cos 90^\circ & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 5 & 7 & 3 \\ 5 & 3 & 3 \\ 1 & 1 & 1 \end{pmatrix} \\
 &= \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 5 & 7 & 3 \\ 5 & 3 & 3 \\ 1 & 1 & 1 \end{pmatrix} \\
 &= \begin{pmatrix} -5 & -3 & -3 \\ 5 & 7 & 3 \\ 1 & 1 & 1 \end{pmatrix}
 \end{aligned}$$

**Note:**  $P' = R \cdot P$



## Exercise 2



**Q.:** Rotate the triangle (5, 5), (7, 3), (3, 3) about fixed point (5, 4) in counter clockwise (CCW) by 90 degree.

**A : Solution**

Here, the required steps are:

1. Translate the fixed point to origin.
2. Rotate about the origin by specified angle  $\theta$ .
3. Reverse the translation as performed earlier.

$$\begin{bmatrix} 1 & 0 & X_f \\ 0 & 1 & Y_f \\ 0 & 0 & 1 \end{bmatrix}$$

$T_x$  = Translate back to fixed point

$$\begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$R_\theta$  = Rotate angle  $\theta$

$$\begin{bmatrix} 1 & 0 & -X_f \\ 0 & 1 & -Y_f \\ 0 & 0 & 1 \end{bmatrix}$$

$T_x$  = Translate fixed point to origin

## Exercise 2

Thus, the composite matrix is given by

$$\text{Com} = T(x_f, y_f) \cdot R_\theta \cdot T(-x_f, -y_f)$$

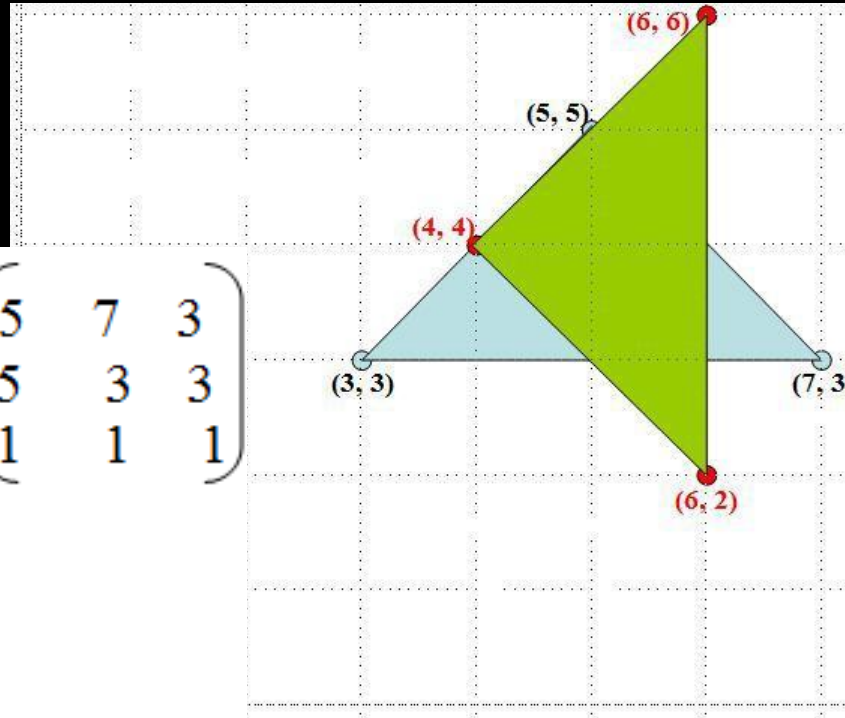
$$\begin{aligned} &= \begin{pmatrix} 1 & 0 & 5 \\ 0 & 1 & 4 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos 90^\circ & -\sin 90^\circ & 0 \\ \sin 90^\circ & \cos 90^\circ & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & -5 \\ 0 & 1 & -4 \\ 0 & 0 & 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 & 5 \\ 0 & 1 & 4 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & -5 \\ 0 & 1 & -4 \\ 0 & 0 & 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 & 5 \\ 0 & 1 & 4 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & -1 & 4 \\ 1 & 0 & -5 \\ 0 & 0 & 1 \end{pmatrix} \\ &= \begin{pmatrix} 0 & -1 & 9 \\ 1 & 0 & -1 \\ 0 & 0 & 1 \end{pmatrix} \end{aligned}$$

## Exercise 2

Now, the required co-ordinate can be calculated as:

$$P' = \text{Com} \cdot P$$

$$= \begin{pmatrix} 0 & -1 & 9 \\ 1 & 0 & -1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 5 & 7 & 3 \\ 5 & 3 & 3 \\ 1 & 1 & 1 \end{pmatrix}$$
$$= \begin{pmatrix} 4 \\ 4 \\ 1 \end{pmatrix} \begin{pmatrix} 6 \\ 6 \\ 1 \end{pmatrix} \begin{pmatrix} 6 \\ 2 \\ 1 \end{pmatrix}$$

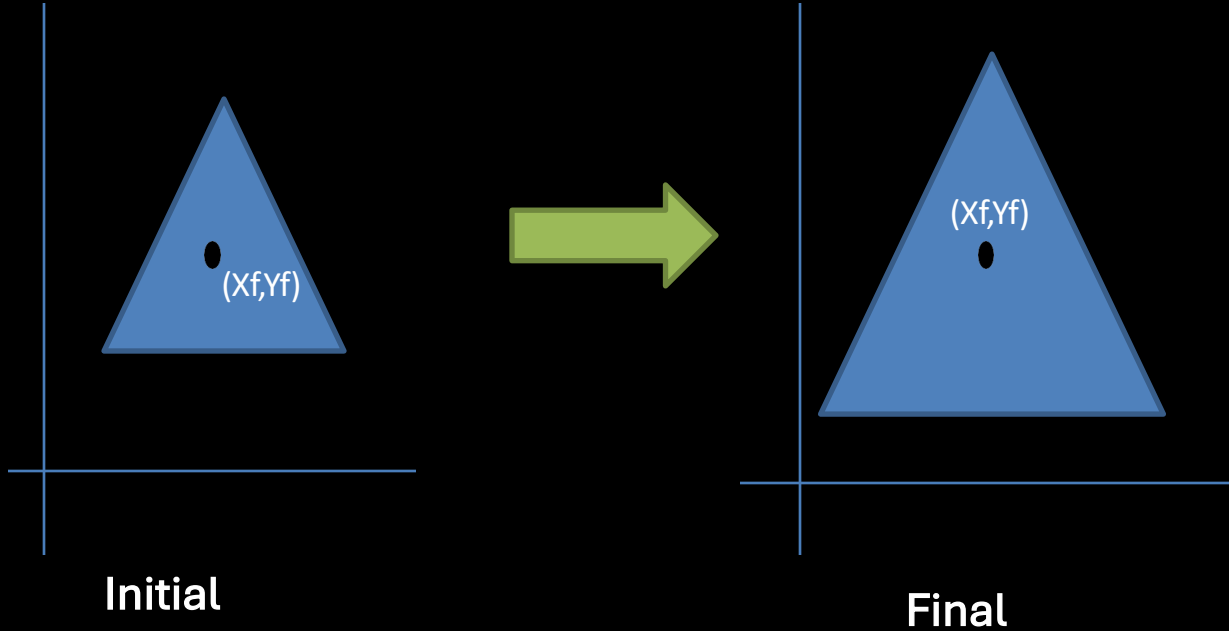


Hence, the new required coordinate points are  
**(4, 4), (6, 6) & (6, 2).**



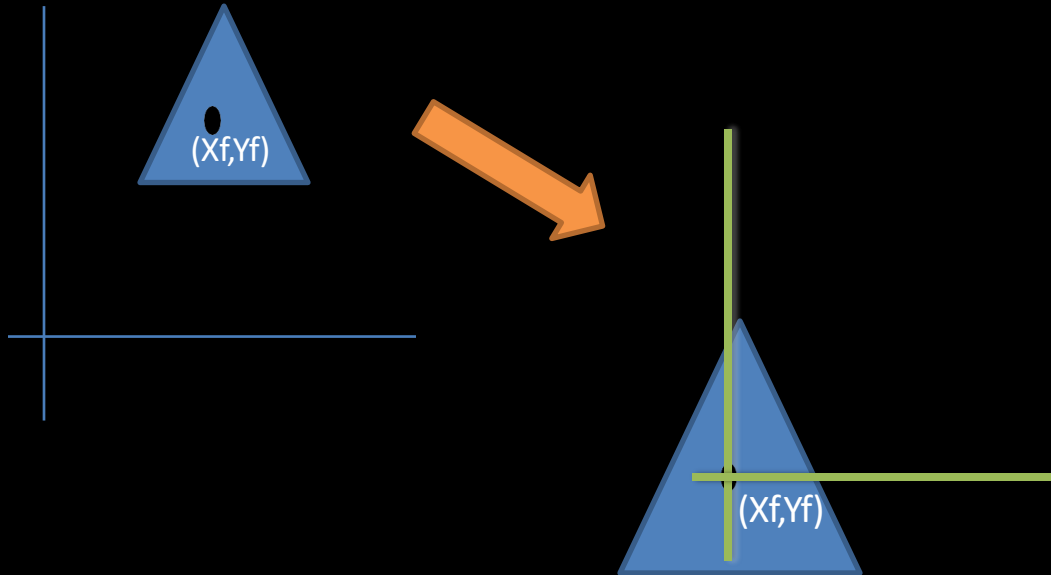
# CASE: Fixed Point Scaling

# Fixed Point Scaling



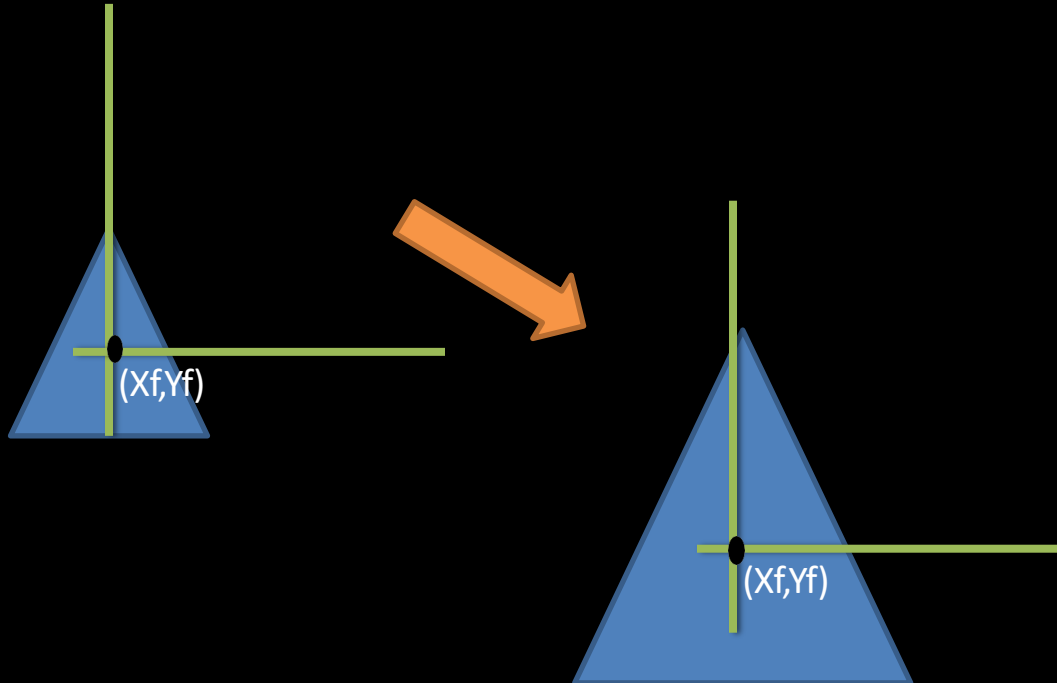
# Fixed Point Scaling

Step 1: The fixed point along with the object is translated to coordinate origin.



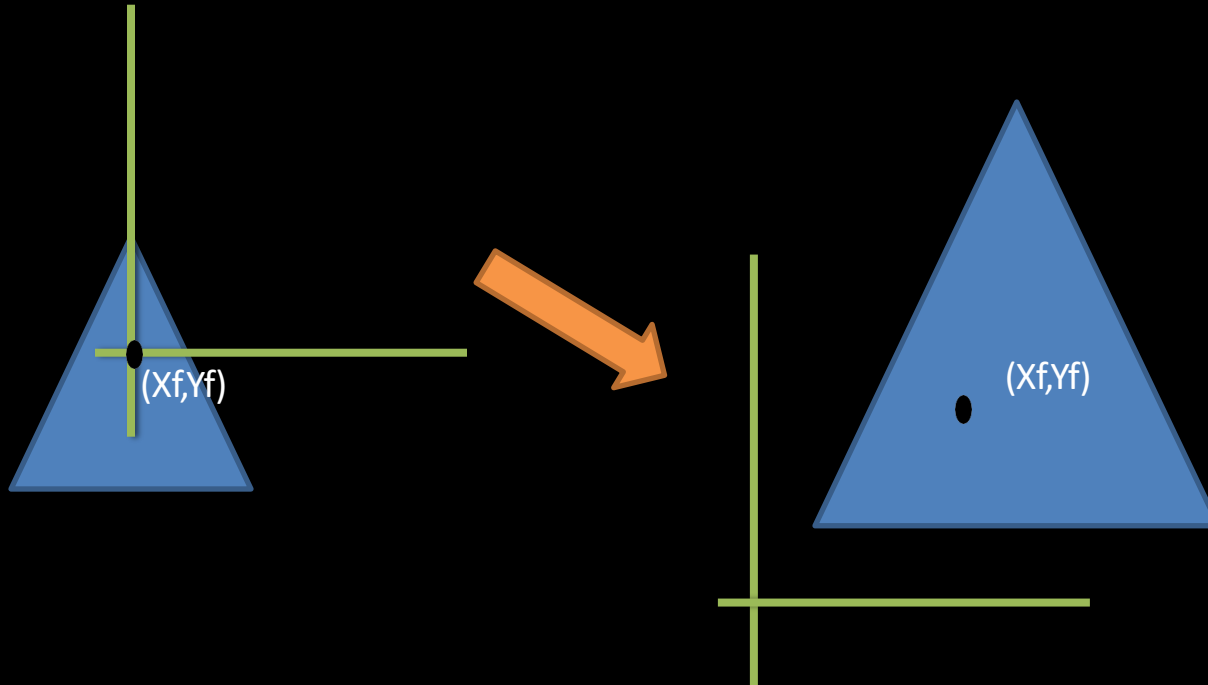
# Fixed Point Scaling

Step 2 : Scaling the object about origin



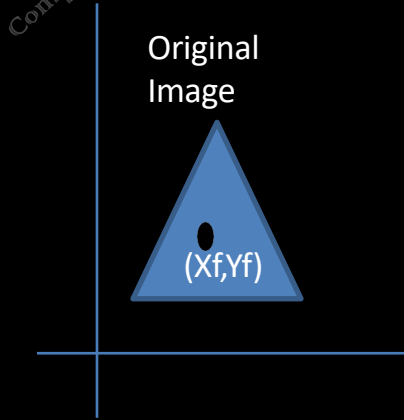
# Fixed Point Scaling

Step 3: The fixed point along with the object is translated back to its original position.

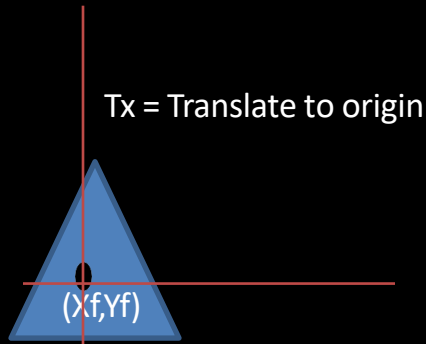




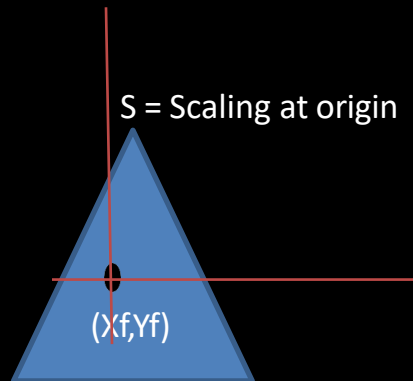
# Fixed Point Scaling



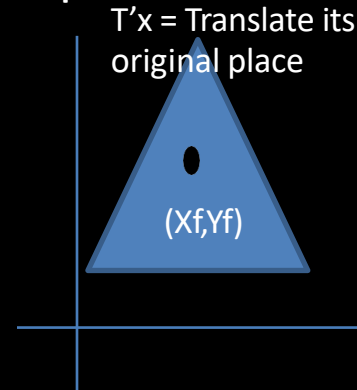
**Step 1**



**Step 2**



**Step 3**



# Fixed Point Scaling

Composite Transformation

$$= T'_{(X_f, Y_f)} \cdot S_{\text{axis}} \cdot T_{(-X_f, -Y_f)}$$

$$= \begin{bmatrix} 1 & 0 & X_f \\ 0 & 1 & Y_f \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -X_f \\ 0 & 1 & -Y_f \\ 0 & 0 & 1 \end{bmatrix}$$

$T'_x$  = Translate back to fixed point

$S$  = Scaling at origin

$T_x$  = Translate to origin

# Fixed Point Scaling

Find the coordinate of a triangle A(1,3) , B(2,5) and C(3,3) after twice its original size

a) about origin

b) about fixed point (2,3)

a) about origin

Given,

A(1,3) , B(2,5) and C(3,3)

In homogeneous form,

$$P = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 5 & 3 \\ 1 & 1 & 1 \end{bmatrix}$$

$$S_x = 2$$

$$S_y = 2$$

Now:  $P' = S \cdot P$

$$= \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 & 3 \\ 3 & 5 & 3 \\ 1 & 1 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 2 & 4 & 6 \\ 6 & 10 & 6 \\ 1 & 1 & 1 \end{bmatrix}$$

# Fixed Point Scaling

Find the coordinate of a triangle A(1,3) , B(2,5) and C(3,3) after twice its original size

a) about origin

b) about fixed point (2,3)

**b) about fixed point (2,3)**

Given,

A(1,3) , B(2,5) and C(3,3)

In homogeneous form,

$$P = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 5 & 3 \\ 0 & 0 & 1 \end{bmatrix}$$

$$S_x = 2$$

$$S_y = 2$$

Steps:

1. Translate to origin from fixed point
2. Scaling at origin
3. Translate back to fixed point

*Now: We need to calculate it in reverse order.*

# Fixed Point Scaling

*Now: We need to calculate it in reverse order.*

$$Com = \begin{bmatrix} 1 & 0 & X_f \\ 0 & 1 & Y_f \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -X_f \\ 0 & 1 & -Y_f \\ 0 & 0 & 1 \end{bmatrix}$$

Translate back to fixed point

Scaling at origin

Translate to origin

$$= \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -2 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 2 & 0 & 2 \\ 0 & 2 & 3 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -2 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 & -2 \\ 0 & 2 & -3 \\ 0 & 0 & 1 \end{bmatrix}$$

Steps:

1. Translate to origin from fixed point
2. Scaling at origin
3. Translate back to fixed point

$$S_x = 2$$

$$S_y = 2$$

# Fixed point Scaling

a) about fixed point(2,3)

Given,

A(1,3) , B(2,5) and C(3,3)

In homogeneous form,

$$P = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 5 & 3 \\ 1 & 1 & 1 \end{bmatrix}$$

$$S_x = 2$$

$$S_y = 2$$

Now:  $P' = \text{Com. } P$

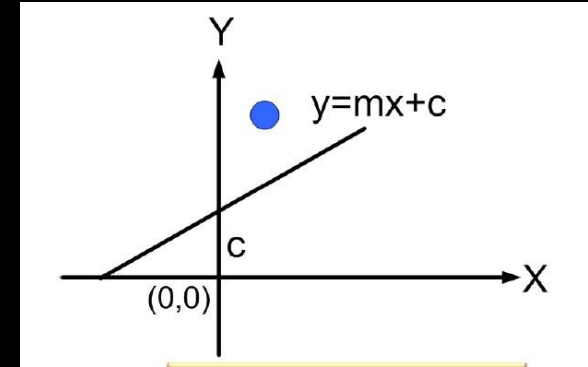
$$= \begin{bmatrix} 2 & 0 & -2 \\ 0 & 2 & -3 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 & 3 \\ 3 & 5 & 3 \\ 1 & 1 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 2 & 4 \\ 3 & 7 & 3 \\ 1 & 1 & 1 \end{bmatrix}$$

A'(0,3) , B'(2,7) and C'(4,3)

# Reflection about Line $y=mx+c$

1. **Translate the line to origin** by decreasing the y-intercept with one.
2. **Rotate the line by angle  $\theta^0$**  in clockwise direction so that the given line must overlap x-axis.
3. **Reflect the object** about the x-axis.
4. **Reverse rotate the line by angle  $-\theta^0$**  in counter-clockwise direction.
5. **Reverse translate the line to original position** by adding the y-intercept with one.



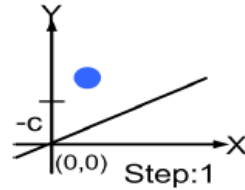
*For composite matrix we need to do reverse of the above steps*

# Reflection about Line $y=mx+c$

1. First translate the line so that it passes through the origin

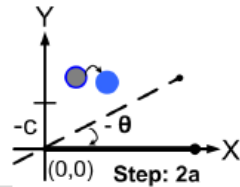
$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & -c \\ 0 & 0 & 1 \end{bmatrix}$$

translation



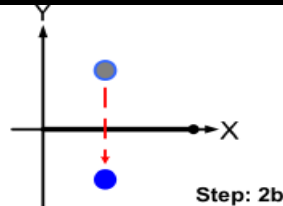
2. Rotate the line onto one of the coordinate axes (say x-axis) and reflect about that axis (x-axis)

$$\begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ --- rotation}$$



$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

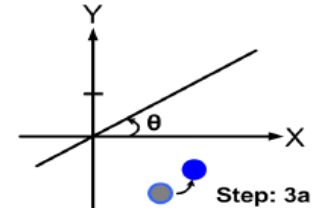
reflection



3. Finally, restore the line to its original position with the inverse rotation and translation transformation.

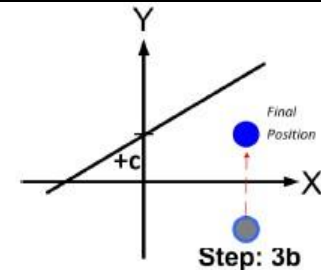
$$\begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

rotation



$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & c \\ 0 & 0 & 1 \end{bmatrix}$$

translation





# Reflection about Line $y=mx+c$

$$CM = T'_{(0,c)} \cdot R'_{\theta} \cdot R_{\text{refl}} \cdot R_{\theta} \cdot T_{(0,-c)}$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & c \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & -c \\ 0 & 0 & 1 \end{bmatrix}$$

## Exercise

**Q.N.1 . Reflect an object (2, 3), (4, 3), (4, 5) about line  $y = x + 1$ .**

Solution

Here,

The given line is  $y = x + 1$ .

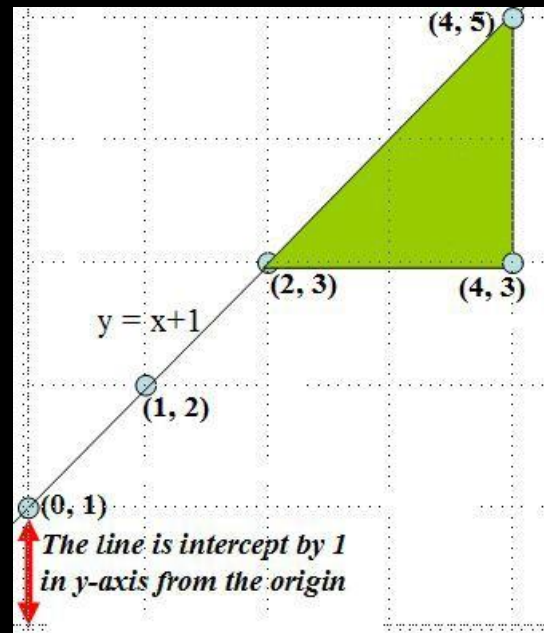
Thus,

When  $x = 0$ ,  $y = 1$  When  $x = 1$ ,  
 $y = 2$  When  $x = 2$ ,  $y = 3$

Also,

The slope of the line ( $m$ ) = 1

Thus, the rotation angle ( $\theta$ ) =  $\tan^{-1}(m) = \tan^{-1}(1) = 45^\circ$



## Exercise

Here, the required steps are:

- Translate the line to origin by decreasing the y-intercept with one.
- Rotate the line by angle  $45^\circ$  in clockwise direction so that the given line must overlap x-axis.
- Reflect the object about the x-axis.
- Reverse rotate the line by angle  $-45^\circ$  in counter-clockwise direction.
- Reverse translate the line to original position by adding the y-intercept with one.

Thus, the composite matrix is given by:

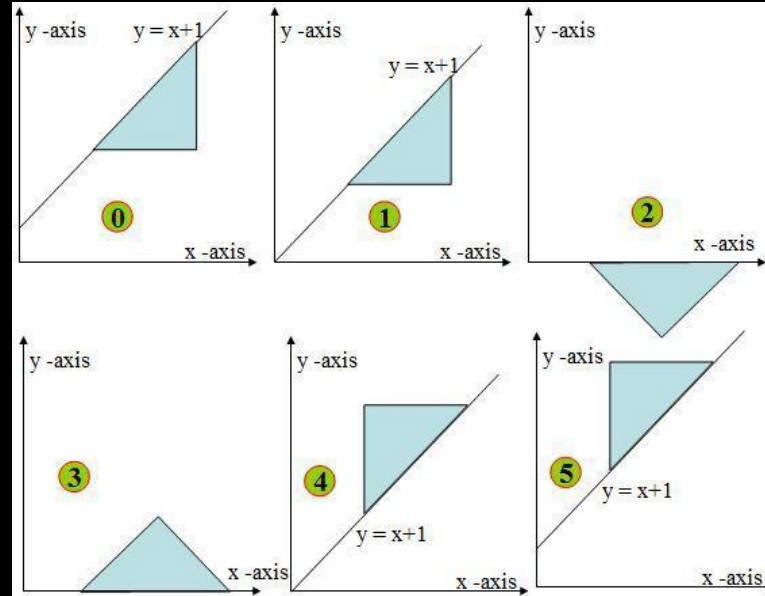
$$CM = T'_{(0,C)} \cdot R'_{\theta} \cdot R_{\text{refl}} \cdot R_{\theta} \cdot T_{(0,-C)}$$



$$\begin{aligned}
 & \begin{array}{c} \text{Addition} \\ \text{y-intercept} \end{array} \begin{array}{c} \text{CCW Rotation} \end{array} \begin{array}{c} \text{Reflection} \\ \text{about x-axis} \end{array} \begin{array}{c} \text{CW Rotation} \end{array} \begin{array}{c} \text{Reduce} \\ \text{y-intercept} \end{array} \\
 & = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos 45^\circ & -\sin 45^\circ & 0 \\ \sin 45^\circ & \cos 45^\circ & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos 45^\circ & \sin 45^\circ & 0 \\ -\sin 45^\circ & \cos 45^\circ & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix} \\
 & = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} & -1/\sqrt{2} & 0 \\ 1/\sqrt{2} & 1/\sqrt{2} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} & 1/\sqrt{2} & 0 \\ -1/\sqrt{2} & 1/\sqrt{2} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix} \\
 & = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} & -1/\sqrt{2} & 0 \\ 1/\sqrt{2} & 1/\sqrt{2} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} & 1/\sqrt{2} & -1/\sqrt{2} \\ -1/\sqrt{2} & 1/\sqrt{2} & -1/\sqrt{2} \\ 0 & 0 & 1 \end{pmatrix} \\
 & = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} & -1/\sqrt{2} & 0 \\ 1/\sqrt{2} & 1/\sqrt{2} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} & 1/\sqrt{2} & -1/\sqrt{2} \\ -1/\sqrt{2} & -1/\sqrt{2} & 1/\sqrt{2} \\ 0 & 0 & 1 \end{pmatrix} \\
 & = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & 1 & -1 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 1 & -1 \\ 1 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix}
 \end{aligned}$$

$$= \begin{pmatrix} 0 & 1 & -1 \\ 1 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 2 & 4 & 4 \\ 3 & 3 & 5 \\ 1 & 1 & 1 \end{pmatrix}$$

$$= \begin{pmatrix} 2 & 2 & 4 \\ 3 & 5 & 5 \\ 1 & 1 & 1 \end{pmatrix}$$



Hence, the final coordinates are (2, 3), (2, 5) & (4, 5).



## Exercise (Imp)

Q.N> A mirror is placed such that it passes through  $(0,10)$ ,  $(10, 0)$ . Find the mirror image of an object  $(6,7)$ ,  $(7, 6)$ ,  $(6, 9)$ .

### Solution

Here,

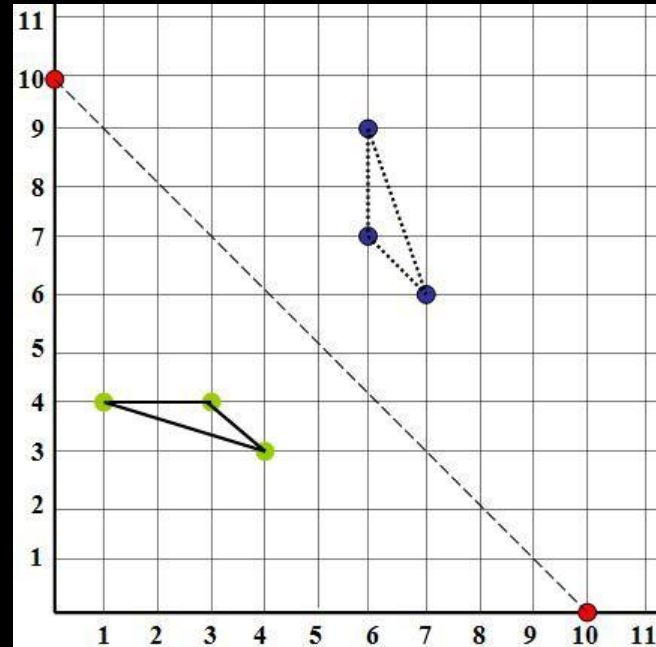
The given mirror or line is passing through the points  $(0, 10)$  &  $(10, 0)$ .

Now, the slope of the line

$$(m) = (y_2 - y_1) / (x_2 - x_1) \\ = (0 - 10) / (10 - 0) = -1$$

Thus, the rotation angle ( $\theta$ )

$$= \tan^{-1}(m) = \tan^{-1}(-1) \\ = -45^\circ$$



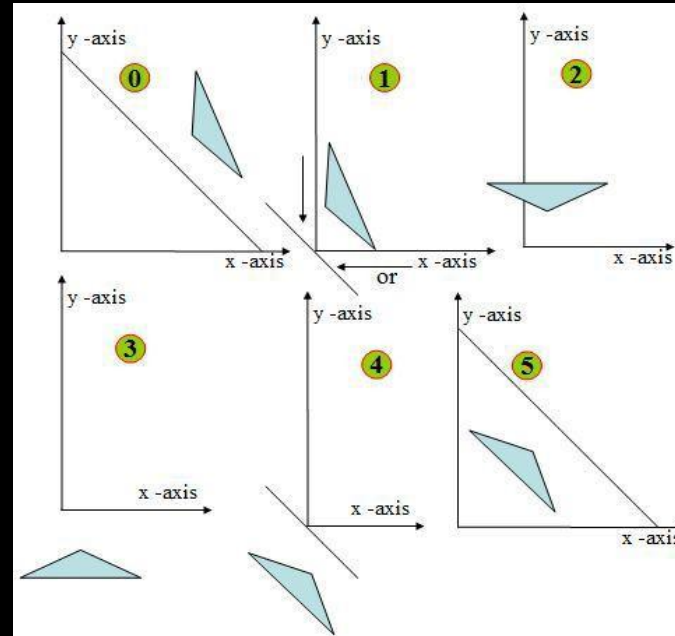
The composite matrix is given by:

$$\text{Com} = T_{(0, 10) \text{ or } (10, 0)} \cdot R_{\theta \text{ in CW}} \cdot R_{fx} \cdot R_{\theta \text{ in CCW}} \cdot T_{(0, -10) \text{ or } (-10, 0)}$$

$$\begin{aligned}
 & \begin{array}{c} \text{Addition} \\ \text{x-intercept} \end{array} \begin{array}{c} \text{CW Rotation} \end{array} \begin{array}{c} \text{Reflection} \\ \text{about x-axis} \end{array} \begin{array}{c} \text{CCW Rotation} \end{array} \begin{array}{c} \text{Reduce} \\ \text{x-intercept} \end{array} \\
 & = \begin{pmatrix} 1 & 0 & 10 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos 45^\circ & \sin 45^\circ & 0 \\ -\sin 45^\circ & \cos 45^\circ & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos 45^\circ & -\sin 45^\circ & 0 \\ \sin 45^\circ & \cos 45^\circ & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & -10 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \\
 & = \begin{pmatrix} 1 & 0 & 10 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} & 1/\sqrt{2} & 0 \\ -1/\sqrt{2} & 1/\sqrt{2} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} & -1/\sqrt{2} & 0 \\ 1/\sqrt{2} & 1/\sqrt{2} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & -10 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \\
 & = \begin{pmatrix} 1 & 0 & 10 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} & 1/\sqrt{2} & 0 \\ -1/\sqrt{2} & 1/\sqrt{2} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} & -1/\sqrt{2} & -10/\sqrt{2} \\ 1/\sqrt{2} & 1/\sqrt{2} & -10/\sqrt{2} \\ 0 & 0 & 1 \end{pmatrix} \\
 & = \begin{pmatrix} 1 & 0 & 10 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} & 1/\sqrt{2} & 0 \\ -1/\sqrt{2} & 1/\sqrt{2} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} & -1/\sqrt{2} & -10/\sqrt{2} \\ -1/\sqrt{2} & -1/\sqrt{2} & 10/\sqrt{2} \\ 0 & 0 & 1 \end{pmatrix} \\
 & = \begin{pmatrix} 1 & 0 & 10 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & -1 & 0 \\ -1 & 0 & 10 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & -1 & 10 \\ -1 & 0 & 10 \\ 0 & 0 & 1 \end{pmatrix}
 \end{aligned}$$

$$= \begin{pmatrix} 0 & -1 & 10 \\ -1 & 0 & 10 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 6 & 7 & 6 \\ 7 & 6 & 9 \\ 1 & 1 & 1 \end{pmatrix}$$

$$= \begin{pmatrix} 3 & 4 & 1 \\ 4 & 3 & 4 \\ 1 & 1 & 1 \end{pmatrix}$$



Hence, the final coordinates are (3, 4), (4, 3) & (1, 4).



## Show that Successive translations are additive

If two successive translations are applied, then they are additive

Proof:

If two successive translation vectors  $(t_{x1}, t_{y1})$  and  $(t_{x2}, t_{y2})$  are applied to a coordinate position P, the final transformed location P' is calculated with the following composite transformation as:

$$\begin{aligned}
 T' &= T_{(x2,y2)} \cdot T_{(x1,y1)} \\
 &= \begin{bmatrix} 1 & 0 & Tx2 \\ 0 & 1 & Ty2 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & Tx1 \\ 0 & 1 & Ty1 \\ 0 & 0 & 1 \end{bmatrix} \\
 &= \begin{bmatrix} 1 & 0 & Tx1 + Tx2 \\ 0 & 1 & Ty1 + Ty2 \\ 0 & 0 & 1 \end{bmatrix}
 \end{aligned}$$

Hence,  $T(t_{x2}, t_{y2}) \cdot T(t_{x1}, t_{y1}) = T(t_{x1}+t_{x2}, t_{y1}+t_{y2})$  which demonstrates that two successive translation are additive

## Show that Successive rotation are additive

If two successive rotations are applied, then they are additive.

Proof:

Let P be point anticlockwise rotated by angle  $\theta_1$  to point P' and again let P' be rotated by angle  $\theta_2$  to point P'', then the combined transformation can be calculated with the following composite matrix as:

$$\begin{aligned}
 T &= R(\theta_2) R(\theta_1) \\
 &= \begin{bmatrix} \cos\theta_2 & -\sin\theta_2 & 0 \\ \sin\theta_2 & \cos\theta_2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\theta_1 & -\sin\theta_1 & 0 \\ \sin\theta_1 & \cos\theta_1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \\
 &= \begin{bmatrix} \cos\theta_2 * \cos\theta_1 - \sin\theta_2 * \sin\theta_1 & -\cos\theta_2 * \sin\theta_1 - \sin\theta_2 * \cos\theta_1 & 0 \\ \sin\theta_2 * \cos\theta_1 + \cos\theta_2 * \sin\theta_1 & -\sin\theta_2 * \sin\theta_1 + \cos\theta_2 * \cos\theta_1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \\
 &= \begin{bmatrix} \cos(\theta_1 + \theta_2) & -\sin(\theta_1 + \theta_2) & 0 \\ \sin(\theta_1 + \theta_2) & \cos(\theta_1 + \theta_2) & 0 \\ 0 & 0 & 1 \end{bmatrix}
 \end{aligned}$$

i.e.  $R_{\theta_2} \cdot R_{\theta_1} = R(\theta_1 + \theta_2)$ , which demonstrates that two successive rotations are additive.

## Show that Successive Scaling are multiplication

If two successive Scaling are applied, then they are multiplicative/commutative

**Proof:**

Let point P is first scaled with scaling factor  $S_{x1}, S_{y1}$  to Point P' and again let P' be scaled by scaling factor  $S_{x2}, S_{y2}$  to Point P'', then the combined transformation can be calculated with the following composite matrix.

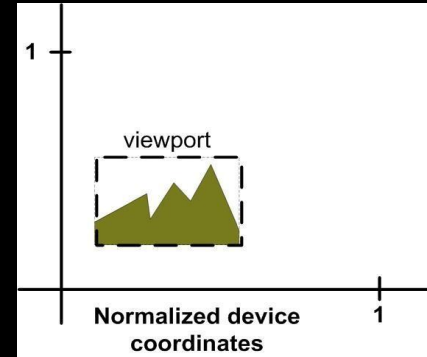
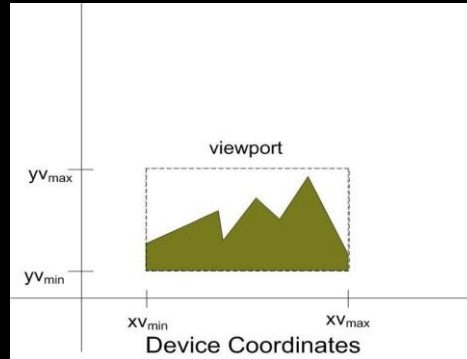
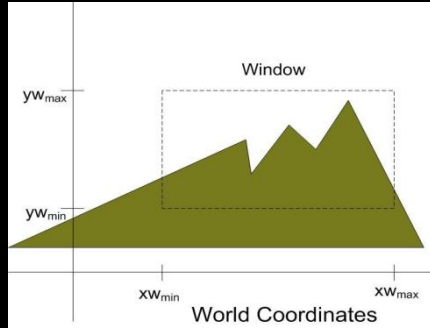
$$T = S(S_{x2}, S_{y2}) S(S_{x1}, S_{y1})$$

$$\begin{bmatrix} S_{x2} & 0 & 0 \\ 0 & S_{y2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} S_{x1} & 0 & 0 \\ 0 & S_{y1} & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} S_{x1}S_{x2} & 0 & 0 \\ 0 & S_{y1}S_{y2} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

i.e.  $S(S_{x2}, S_{y2}) S(S_{x1}, S_{y1}) = S(S_{x1}S_{x2}, S_{y1}S_{y2})$ , which demonstrates that two successive scaling are multiplicative.

## **Section 3: Windows to View port transformation, 2D Viewing pipeline**

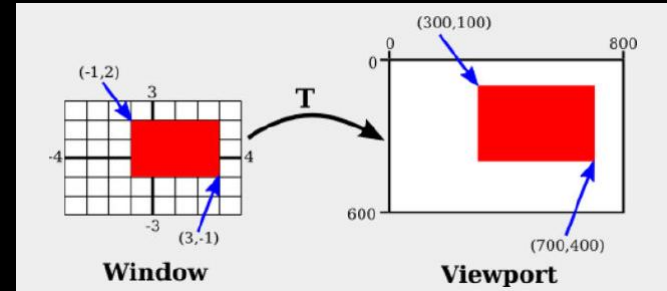
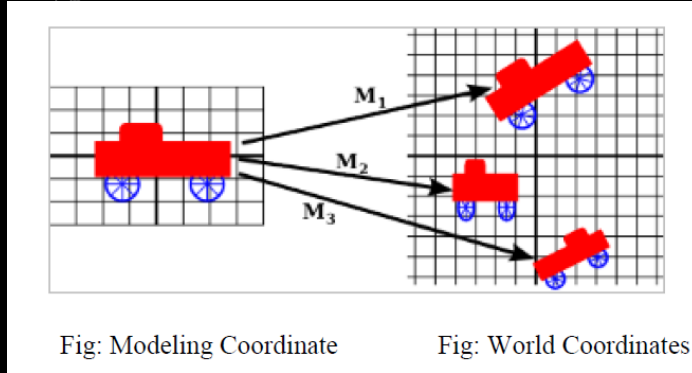
# Viewing Transformation: Transformation from world to viewport



## Model Coordinates

The Model Coordinate System is simply the coordinate system where the model was created. It is used to define coordinates that are used to construct the shape of individual parts (objects) of a 2D scene.

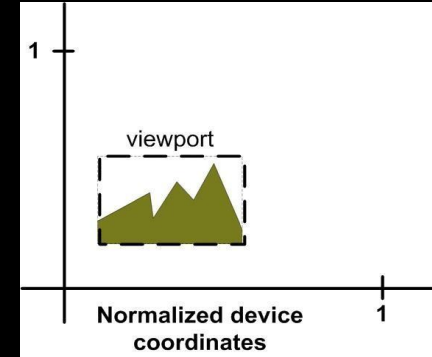
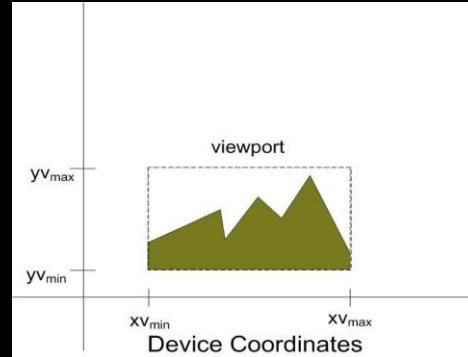
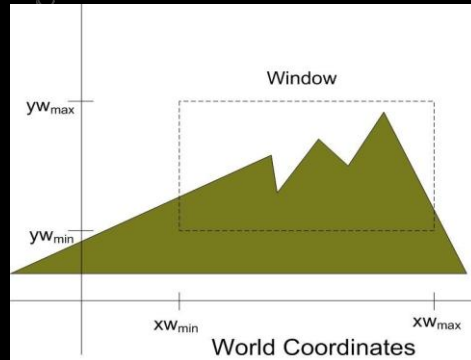
# Viewing Transformation: Transformation from world to viewport



**World Coordinates:** World coordinates, also known as world space or modeling coordinates, refer to the coordinate system that is used to define the positions and shapes of objects in a three-dimensional scene.

**Device Coordinates:** Device co-ordinate are used to define coordinates in an output device. They are integers within the range  $(0, 0)$  to  $(x_{\max}, y_{\max})$  for a particular output device.

# Viewing Transformation: Transformation from world to viewport



**Normalized Device Coordinates:** Normalized Device Coordinates are a standardized coordinate system that is derived from device coordinates. In this system, the device coordinates are normalized to a common range, typically from -1 to 1 for both x and y axes.

# 2D Viewing Pipeline



## 2-D Viewing pipeline

The process of mapping the world co-ordinate scene to device co-ordinate is called viewing transformation or window to view port transformation or windowing transformation.

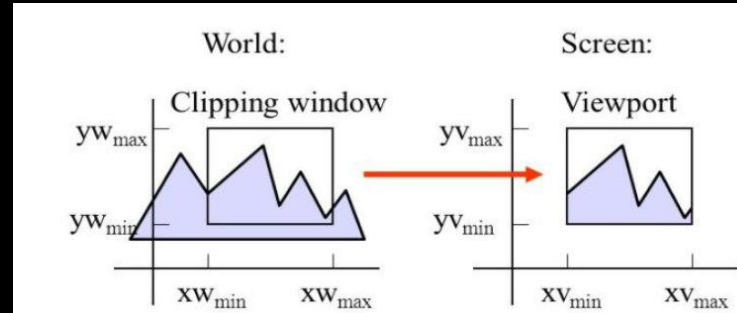
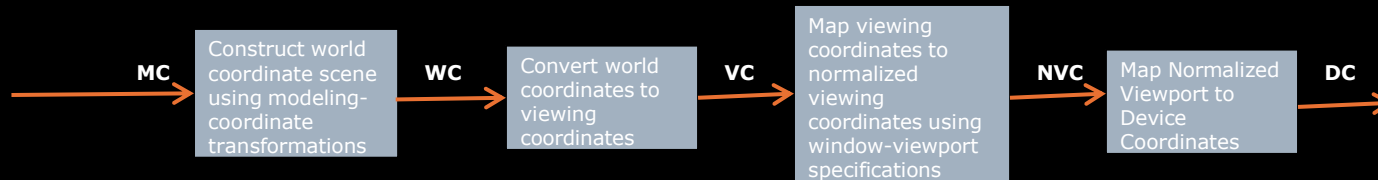
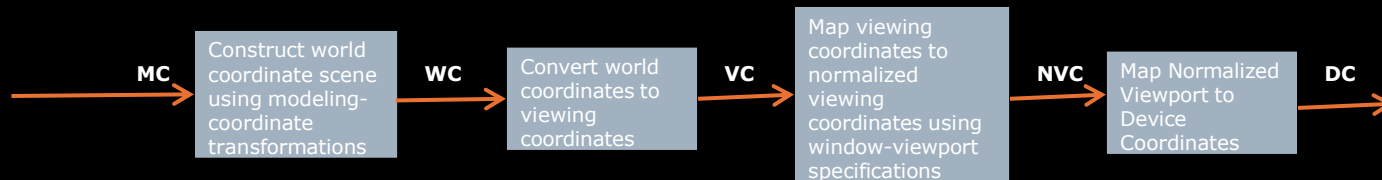


Fig. 1. Mapping of picture section falling under rectangular Window onto a designated rectangular view port



## 2-D Viewing pipeline

- ✓ The window defines what is to be viewed whereas the view port defines where it is to be displayed. Window can have any shape (circle, polygon) however some graphics package provide window and view port operations on standard rectangle only.
- ✓ Window deals with object space whereas view port deals with image space.
- ✓ Transformations from world to device coordinates involve translation, rotation and scaling operations, as well as procedures for deleting those parts of the picture that are outside the limits of a selected display area i.e. clipping.
- ✓ To make the viewing process independent of the requirements of any output device, graphics systems convert object descriptions to normalized coordinates.



## 2-D Viewing pipeline

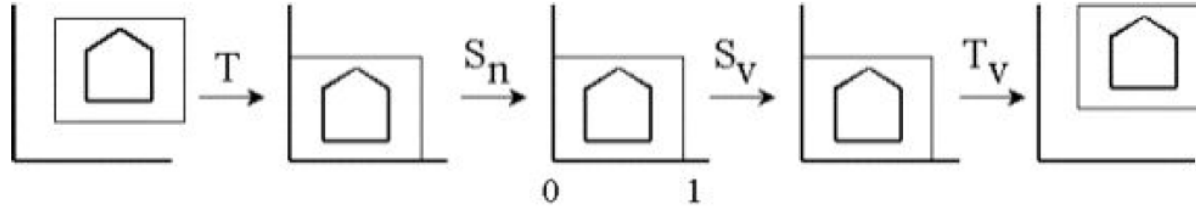
Procedure for to transform a window to the view port we have to perform the following steps:

**Step1:** The object together with its window is translated until the lower left corner of the window is at the origin

**Step2:** The object and window are scaled until the window has the dimensions of the view port

**Step3:** Again translate to move the view port to its correct position on the screen

(Setup Window, Translate window, Scale to normalize, Scale to view port, Translate to View port)



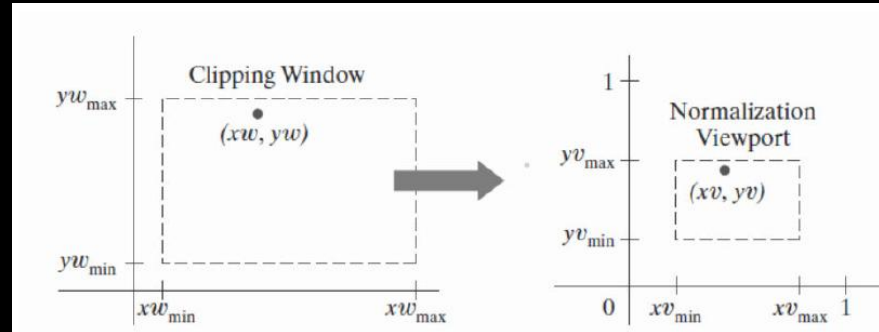
### Application

- By changing the position of the view port, we can view objects at different positions on the display area of an output device.
- By varying the size of view ports, we can change size of displayed objects.
- Zooming effects can be obtained by successively mapping different-sized windows on a fixed-sized view port
- Panning effects (Horizontal Scrolling) are produced by moving a fixed-size window across the various objects in a scene.



## 2-D Viewing pipeline: Window to View port Coordinate Transformation

A window can be specified by four world coordinates:  $xw_{\min}$ ,  $xw_{\max}$ ,  $yw_{\min}$  and  $yw_{\max}$ . Similarly, a view port can be described by four device coordinates:  $xv_{\min}$ ,  $xv_{\max}$ ,  $yv_{\min}$  and  $yv_{\max}$ .



The following proportional ratios must be equal.

$$\frac{xv - xv_{\min}}{xv_{\max} - xv_{\min}} = \frac{xw - xw_{\min}}{xw_{\max} - xw_{\min}}$$
$$\frac{yv - yv_{\min}}{yv_{\max} - yv_{\min}} = \frac{yw - yw_{\min}}{yw_{\max} - yw_{\min}}$$

## 2-D Viewing pipeline: Window to View port Coordinate Transformation

Solving these expressions, we get

$$x_v = x_{v_{\min}} + (x_w - x_{w_{\min}})S_x$$

$$y_v = y_{v_{\min}} + (y_w - y_{w_{\min}})S_y$$

where,

$$S_x = \frac{x_{v_{\max}} - x_{v_{\min}}}{x_{w_{\max}} - x_{w_{\min}}}$$

$$S_y = \frac{y_{v_{\max}} - y_{v_{\min}}}{y_{w_{\max}} - y_{w_{\min}}}$$

**NOTE:**

( $x_{w_{\min}}$ ,  $y_{w_{\min}}$ ,  $x_{w_{\max}}$ ,  $y_{w_{\max}}$ )

( $x_{v_{\min}}$ ,  $y_{v_{\min}}$ ,  $x_{v_{\max}}$ ,  $y_{v_{\max}}$ )

## 2-D Viewing pipeline: Window to View port Coordinate Transformation

### Question:

Window port is given by (100, 100, 300, 300) and view port is given by (50,50,150,150). Convert the window port coordinate (200, 200) to the view port coordinate.

### Solution:

$$\begin{aligned}
 (XW_{\min}, yW_{\min}) &= (100, 100) \\
 (XW_{\max}, yW_{\max}) &= (300, 300) \\
 (XV_{\min}, yV_{\min}) &= (50, 50) \\
 (XV_{\max}, yV_{\max}) &= (150, 150) \\
 (xw, yw) &= (200, 200)
 \end{aligned}$$

Now,

$$\begin{aligned}
 S_x &= \frac{xv_{\max} - xv_{\min}}{xw_{\max} - xw_{\min}} = (150 - 50) / (300 - 100) = 0.5 \\
 S_y &= \frac{yv_{\max} - yv_{\min}}{yw_{\max} - yw_{\min}} = (150 - 50) / (300 - 100) = 0.5
 \end{aligned}$$

## 2-D Viewing pipeline: Window to View port Coordinate Transformation

The equations for mapping window co-ordinate to view port coordinate is given by

$$xv = xv_{\min} + (xw - xw_{\min})S_x$$

$$yv = yv_{\min} + (yw - yw_{\min})S_y$$

Hence,

$$xv = 50 + (200 - 100) * 0.5 = 100$$

$$yv = 50 + (200 - 100) * 0.5 = 100$$

The transformed view port coordinate is (100,100).

Question:

The window port is given by (20,40,80,80) and the view port is given by (30,40,60,60). Convert the window port coordinate (30, 80) to the view port coordinate.

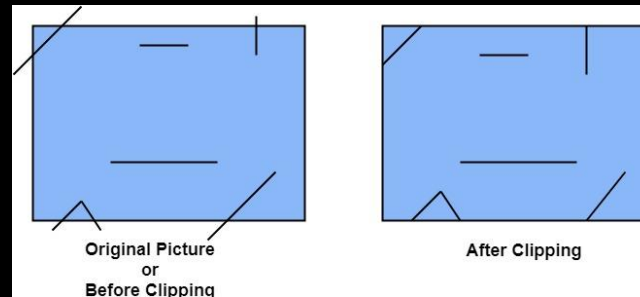
## Section 4: Clipping, Cohen Sutherland Line Clipping Algorithm



# Clipping

# Clipping

- ✓ The process of discarding those parts of a picture which are outside of a specified region or window is called **clipping**.
- ✓ The procedure using which we can identify whether the portions of the graphics object is within or outside a specified region or space is called **clipping algorithm**.
- ✓ The region or space which is used to see the object is called window and the region on which the object is shown is called **view port**.



## Applications of Clipping

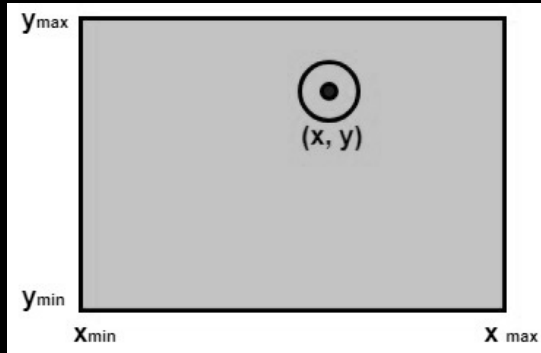
- i. Extracting part of a defined scene for viewing.
- ii. Identifying visible surfaces in three-dimensional views.
- iii. Drawing and painting operations that allow parts of a picture to be selected for copying, moving erasing, or duplicating.

## Types of Clipping

- i. Point Clipping
- ii. Line Clipping (straight-line segments)
- iii. Area Clipping (polygons)
- iv. Curve Clipping
- v. Text Clipping

## Clipping: Point Clipping

- **Point clipping** is the process of removing all those points that lie outside a given region or window.
- Let  $x_{\min}$  and  $x_{\max}$  be the edge of the boundaries of the clipping window parallel to Y axis and  $y_{\min}$  and  $y_{\max}$  be the edge of the boundaries of the clipping window parallel to X axis as shown in figure below.

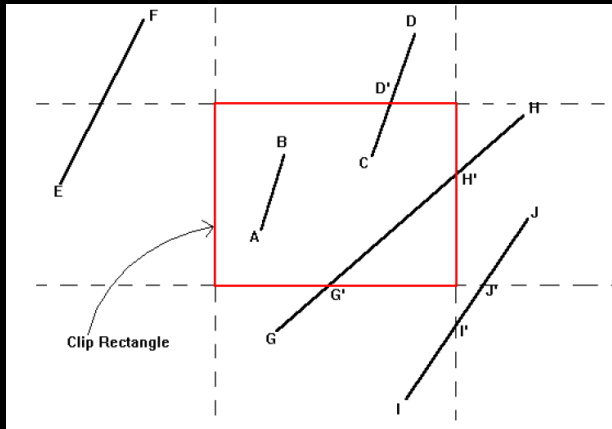


So, any point  $P(x, y)$  of world coordinate can be saved for display if the following conditions are satisfied.

1.  $x \leq x_{\max}$
2.  $x \geq x_{\min}$
3.  $y \leq y_{\max}$
4.  $y \geq y_{\min}$

## Clipping: Line Clipping ( Straight Line Segments)

- In line clipping, a line or part of line is clipped if it is outside the window port. All the line segment falls into one of the following clipping cases.
  1. The line is totally outside the window port so is directly clipped.
  2. The line is partially clipped if a part of the line lies outside the window port.
  3. The line is not clipped if all whole line lied within the window port.



So, in the above figure, line FE require total clipping, CD, GH require partial clipping and AB requires no clipping.

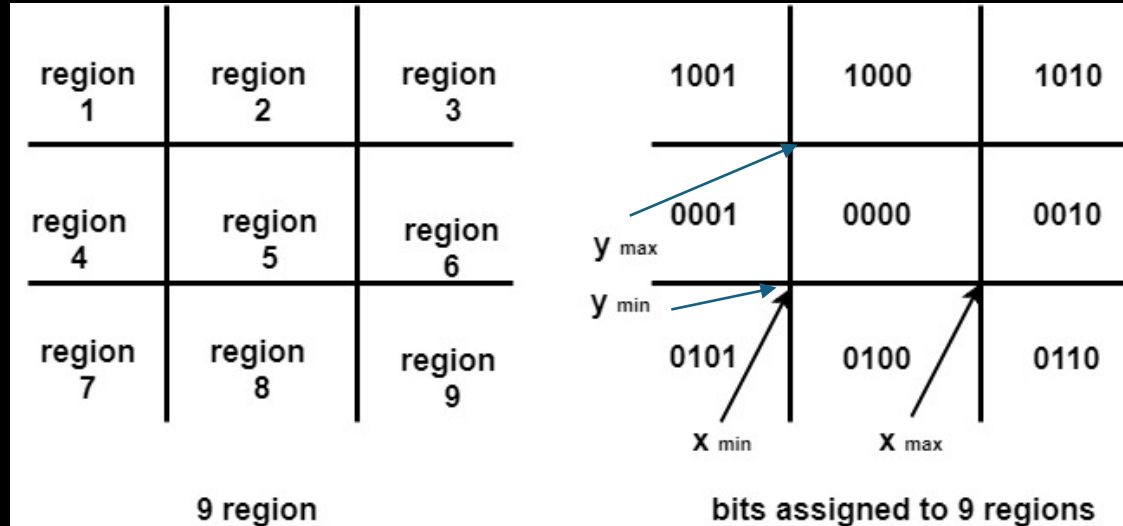
# Clipping: Cohen Sutherland Line Clipping Algorithm

## Cohen Sutherland Line Clipping Algorithm

- The Cohen-Sutherland algorithm is a line-clipping algorithm used in computer graphics to clip a line segment against a rectangular viewing window.
- This algorithm divides the 2D space into nine regions and efficiently determines whether a line segment lies entirely inside, outside, or partially inside the window.
- The nine regions are defined by three horizontal and three vertical lines that divide the space into a 3x3 grid.

|             |             |             |
|-------------|-------------|-------------|
| region<br>1 | region<br>2 | region<br>3 |
| region<br>4 | region<br>5 | region<br>6 |
| region<br>7 | region<br>8 | region<br>9 |

# Clipping: Cohen Sutherland Line Clipping Algorithm

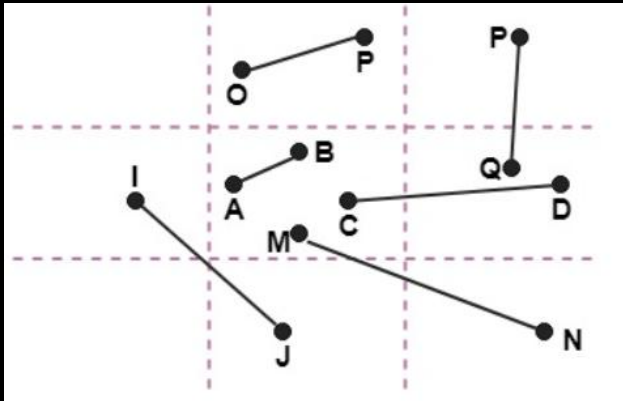


|     |        |       |      |
|-----|--------|-------|------|
| 4   | 3      | 2     | 1    |
| Top | Bottom | Right | Left |

## Clipping: Cohen Sutherland Line Clipping Algorithm

Three cases:

1. Visible Case
2. Invisible Case
3. Clipping candidate



- Line AB is the visible case
- Line OP is an invisible case
- Line PQ is an invisible line
- Line MN are clipping candidate
- Line CD are clipping candidate

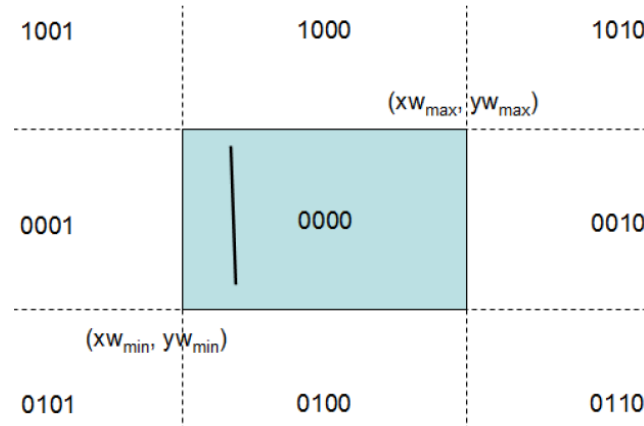


# Clipping: Cohen Sutherland Line Clipping Algorithm

1. Given a line segment with end points  $P1=(x1,y1)$  and  $P2(x2,y2)$ , compute 4 bit region code for each end point.
2. To clip a line, first we have to find out which regions its two endpoints lie in.

## Case I:

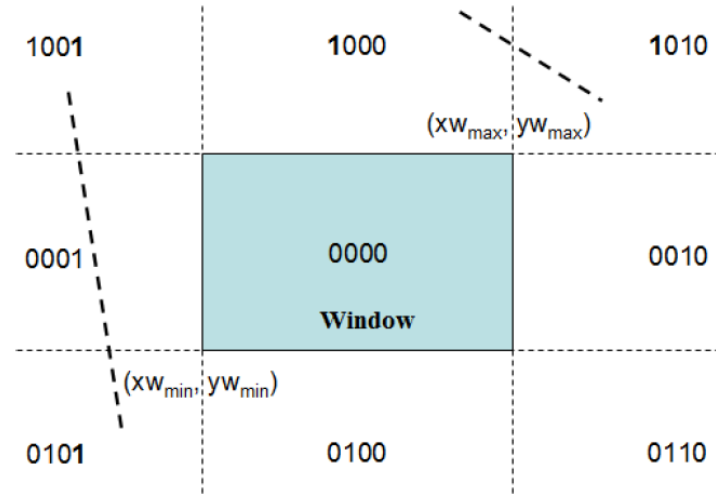
If both end point code is 0000, then the line segment is completely inside the window. i.e. if bitwise OR of the codes yields 0000, then the line segment is accepted for display.



# Clipping: Cohen Sutherland Line Clipping Algorithm

## Case II:

If the two end point region code both have a 1 in the same bit position, then the line segment lies completely outside the window, so discarded. i.e. if bitwise AND of the codes not equal to 0000, then the line segment is rejected.



## Clipping: Cohen Sutherland Line Clipping Algorithm

### Case III:

If lines cannot be identified as completely inside or outside we have to do some more calculations.

Here we find the intersection points with a clipping boundary using the slope intercept form of the line equation

Here, to find the visible surface, the intersection points on the boundary of window can be determined as:

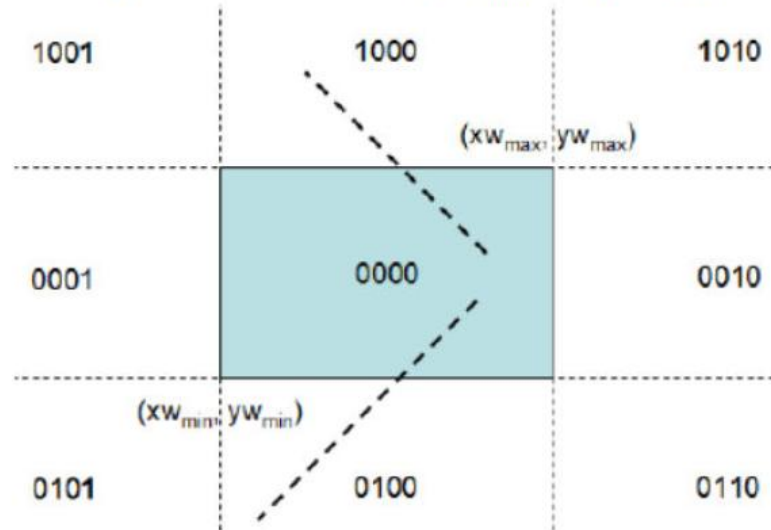
$$y - y_1 = m(x - x_1)$$
$$\text{where } m = (y_2 - y_1) / (x_2 - x_1)$$

# Clipping: Cohen Sutherland Line Clipping Algorithm

- i. If the intersection is with horizontal boundary

$$x = x_1 + (y - y_1)/m$$

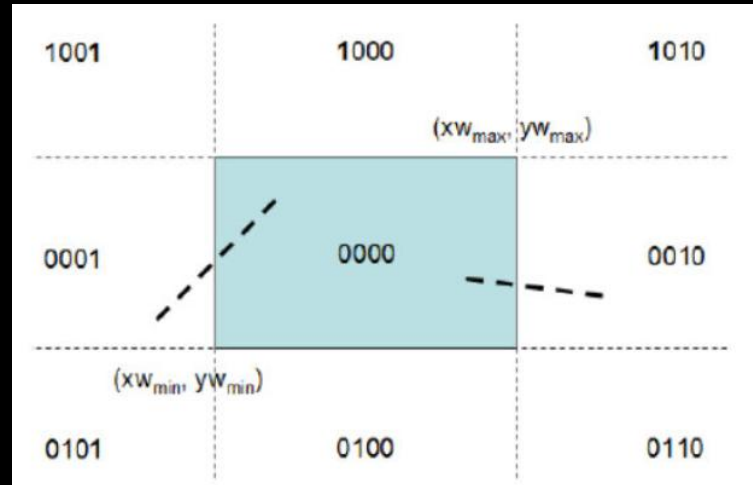
Where  $y$  is set to either  $y_{w_{min}}$  or  $y_{w_{max}}$



# Clipping: Cohen Sutherland Line Clipping Algorithm

- ii. If the intersection is with vertical boundary  
 $y = y_1 + m (x - x_1)$

Where  $x$  is set to either  $Xw_{\min}$  or  $Xw_{\max}$

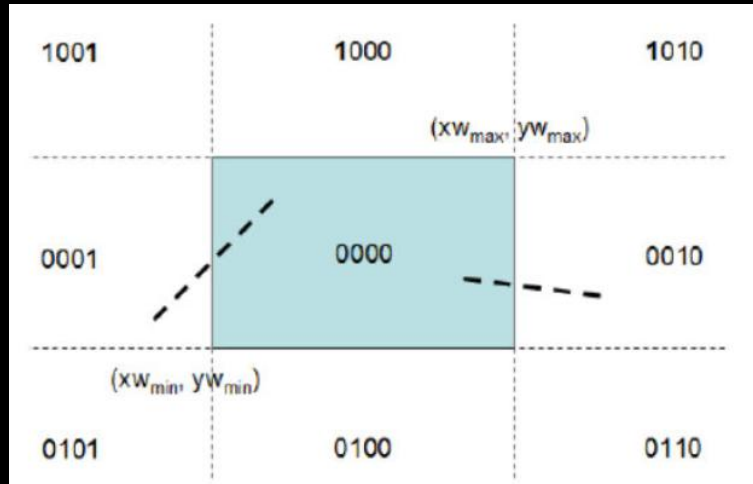


# Clipping: Cohen Sutherland Line Clipping Algorithm

ii. If the intersection is with vertical boundary

$$y = y_1 + m (x - x_1)$$

Where x is set to either  $X_{w_{min}}$  or  $X_{w_{max}}$



## Note:

**Visible:** The line is visible if four bit region code is zero.

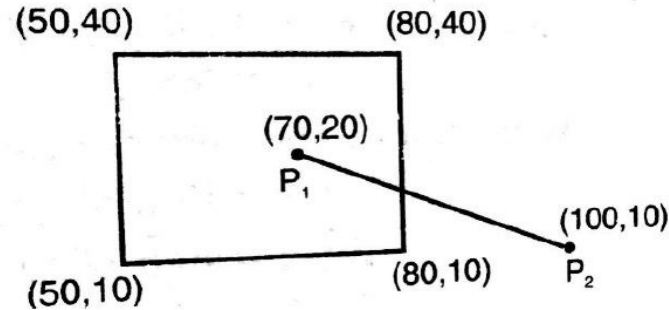
**Not visible:** The line is not visible if bit wise AND Product of code is not zero.

**Needs Clipping:** The line would be clipped if bitwise AND product of code is zero

- Assign a new four-bit code to the intersection point and repeat until either case 1 or case 2 are satisfied.

**Question:** Use the Cohen - Sutherland algorithm to clip the line  $P1(70,20)$  and  $P2(100,10)$  against a window lower left-hand corner  $(50,10)$  and upper right-hand corner  $(80,40)$ .

Use the Cohen - Sutherland algorithm to clip the line  $P_1(70,20)$  and  $P_2(100,10)$  against a window lower left-hand corner  $(50,10)$  and upper right-hand corner  $(80,40)$ .



Assigning 4 bit binary code to the two end point

$$P_1 = 0000$$

$$P_2 = 0010$$

**Finding bitwise OR:**

$$P_1 \mid P_2 = 0000 \mid 0010 = 0010$$

Since  $P_1 \mid P_2 \neq 0000$ , hence the two point doesn't lie completely inside the window.

**Finding bitwise AND:**

$$P_1 \& P_2 = 0000 \& 0010 = 0000$$

since  $P_1 \& P_2 = 0000$ , hence line is partially visible.





**Use the Cohen - Sutherland algorithm to clip the line P1(70,20) and P2(100,10) against a window lower left-hand corner (50,10) and upper right-hand corner (80,40).**

Now, For finding the intersection of P1 and P2 with the boundary of Window,

We have,

$$P1(x_1, y_1) = (70, 20)$$

$$P2(x_2, y_2) = (100, 10)$$

$$\text{Slope } m = (10 - 20) / (100 - 70) = -1/3$$

We have to find the intersection with right edge of window, here  $x=80$ ,  $y=?$

We have

$$\begin{aligned} y &= y_2 + m(x - x_2) \\ &= 10 + (-1/3)(80 - 100) \\ &= 10 + 6.67 \\ &= 16.67 \end{aligned}$$

Thus the intersection point  $P3 = (80, 16.66)$ , So discarding the line segment that lie outside the boundary i.e  $P3P2$ , we get new line  $P1P3$  with co-ordinate  $P1(70, 20)$  and  $P3(80, 16.67)$

## Pros and cons of Cohen Sutherland Line Clipping Algorithm

### Pros:

- It calculates end-points very quickly and rejects and accepts lines quickly.
- It can clip pictures much large than screen size.

### Cons:

- It can be used only on a rectangular clip window.
- It is slow as compare to other approach.

## **Section 5: Polygon Clipping Cohen Sutherland-Hodgeman**

## Polygon Clipping: Sutherland – Hodgeman

- The Sutherland–Hodgeman algorithm is used for clipping polygons. A single polygon can actually be split into multiple polygons. The algorithm clips a polygon against all edges of the clipping region in turn.
- This algorithm is actually quite general - the clip region can be any convex polygon in 2D, or any convex polyhedron in 3D.

There are four possible cases for any given edge of given polygon against clipping edge.

### 1. **Both vertices are inside :**

- Only the second vertex is added to the output list

### 2. **First vertex is outside while second one is inside :**

- Both the point of intersection of the edge with the clip boundary and the second vertex are added to the output list

## Polygon Clipping: Sutherland – Hodgeman

1. Construct an input vector list ( $v_0, v_1, v_2, \dots, v_n$ ) where  $v_0 = v_n$
2. Create empty output vertex list.
3. For each pair of adjacent vertices  $v_i$  and  $v_{i+1}$  perform the following inside-outside test.
  - i. If out – in ( $v_i$  is outside the window boundary,  $v_{i+1}$  is inside), add insertion point  $v_i'$  (new vertex) and  $v_{i+1}$  to the output vertex list.
  - ii. If in-in (both  $v_i, v_{i+1}$  are inside the window boundary), add  $v_{i+1}$  to output vertex list.

## Polygon Clipping: Sutherland – Hodgeman

- iii. If in – out ( $v_i$  is inside the window boundary and  $v_{i+1}$  is outside), add insertion point  $v_i'$  (new vertex) to output vertex list.
- iv. If out – out (both  $v_i, v_{i+1}$  are outside the window boundary), add nothing to the out put vertex list.

| Case | 1st vertex | 2nd vertex | output               |
|------|------------|------------|----------------------|
| 1    | inside     | inside     | 2nd vertex           |
| 2    | inside     | outside    | intersection         |
| 3    | outside    | outside    | none                 |
| 4    | outside    | inside     | 2nd and intersection |

### Example:

Clip polygon ABCD against window PQRS. The Co-ordinates of the polygon are A(80, 200), B(220, 120), C(150, 100), D(100, 30), E(10, 120). Co-ordinates of the window are P(200, 50), Q(200, 150), R (50, 150), S(50, 50).

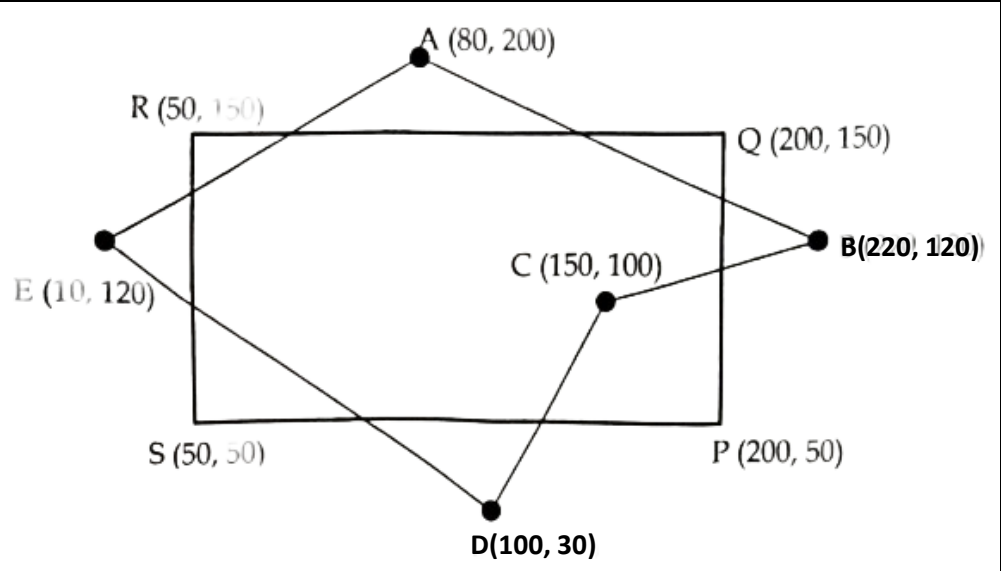
## Example: Solution – Step 1

The Co-ordinates of the polygon are

A(80, 200),  
B(220, 120),  
C(150, 100),  
D(100, 30), E(10, 120).

Co-ordinates of the window are

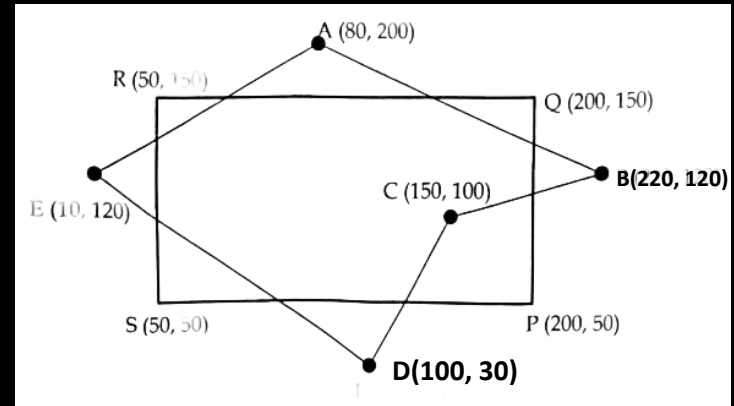
P(200, 50),  
Q(200, 150),  
R(50, 150),  
S(50, 50).





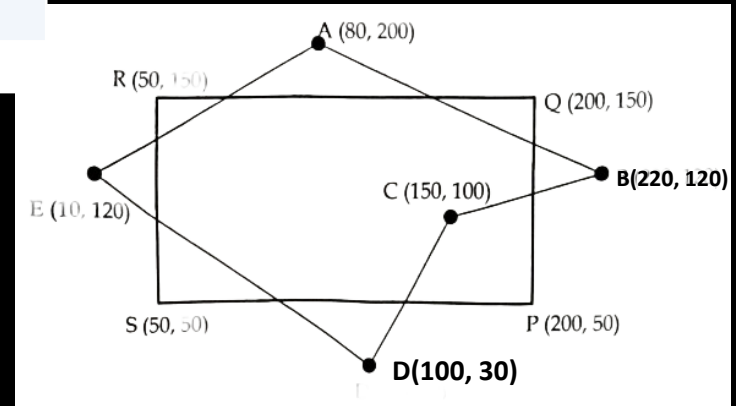
## Step 2: Clipping against left edge of the window (Left Clipping)

| Vertex | Case                 | Output Vertex | Remarks    |
|--------|----------------------|---------------|------------|
| AB     | In $\rightarrow$ In  | B             |            |
| BC     | In $\rightarrow$ In  | C             |            |
| CD     | In $\rightarrow$ In  | D             |            |
| DE     | In $\rightarrow$ Out | D'            | New Vertex |
| EA     | Out $\rightarrow$ In | E'A           | New Vertex |



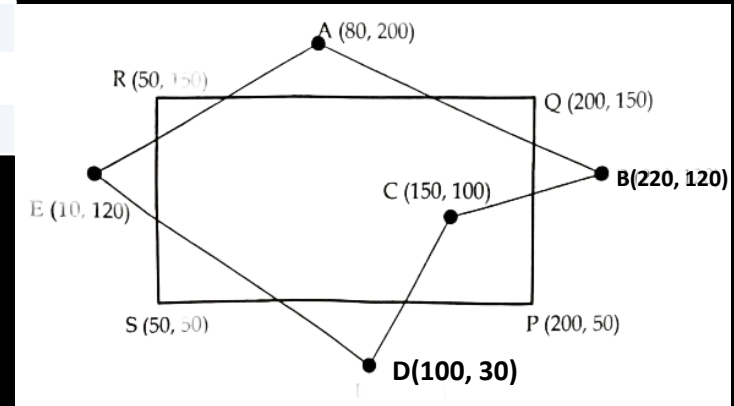
### Step 3: Right Clipping

| Vertex | Case                 | Output Vertex | Remarks    |
|--------|----------------------|---------------|------------|
| AB     | In $\rightarrow$ Out | A'            | New Vertex |
| BC     | Out $\rightarrow$ In | B'C           | New Vertex |
| CD     | In $\rightarrow$ In  | D             |            |
| DD'    | In $\rightarrow$ In  | D'            |            |
| D'E'   | In $\rightarrow$ In  | E'            |            |
| E'A    | In $\rightarrow$ In  | A             |            |



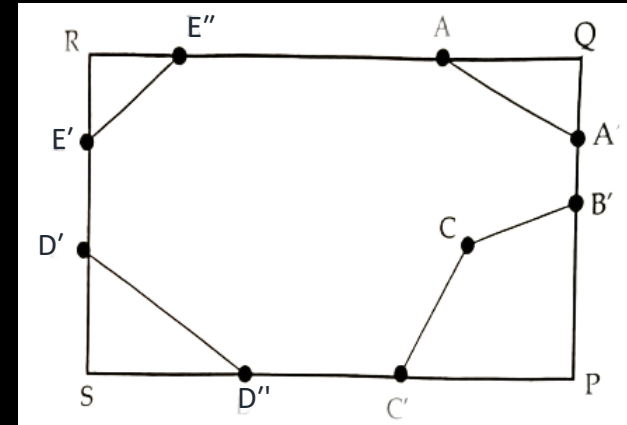
## Step 4: Bottom Clipping

| Vertex | Case                 | Output Vertex | Remarks    |
|--------|----------------------|---------------|------------|
| AA'    | In $\rightarrow$ In  | A'            |            |
| A'B'   | In $\rightarrow$ In  | B'            |            |
| B'C    | In $\rightarrow$ In  | C             |            |
| CD     | In $\rightarrow$ Out | C'            | New Vertex |
| DD'    | Out $\rightarrow$ In | D'D           | New Vertex |
| D'E'   | In $\rightarrow$ In  | E'            |            |
| E'A    | In $\rightarrow$ In  | A             |            |



## Step 5: Top Clipping

| Vertex | Case                 | Output Vertex | Remarks    |
|--------|----------------------|---------------|------------|
| AA'    | Out $\rightarrow$ In | A''           | New Vertex |
| A'B'   | In $\rightarrow$ In  | B'            |            |
| B'C    | In $\rightarrow$ In  | C             |            |
| CC'    | In $\rightarrow$ In  | C'            |            |
| C'D'   | In $\rightarrow$ In  | D''           |            |
| D'D'   | In $\rightarrow$ In  | D'            |            |
| D'E'   | In $\rightarrow$ In  | E'            |            |
| E'A    | In $\rightarrow$ Out | E''           | New Vertex |



## Section 6: Liang Barsky Algorithm

# Liang - Barsky algorithm

- The Liang-Barsky algorithm is a line clipping algorithm.
- This algorithm is more efficient than Cohen-Sutherland line clipping algorithm and can be extended to 3-Dimensional clipping.
- This algorithm is considered to be the faster parametric line-clipping algorithm.
- The following concepts are used in this clipping:
  - The parametric equation of the line.
  - The inequalities describing the range of the clipping window which is used to determine the intersections between the line and the clip window.

## Liang - Barsky algorithm

This algorithm uses the parametric equations for a line and solves four inequalities to find the range of the parameter for which the line is in the viewport.

Parametric Equations are given below:

$$x = x_1 + t \Delta x$$

$$y = y_1 + t \Delta y, \quad 0 \leq t \leq 1$$

Where  $\Delta x = x_2 - x_1$   $\Delta y = y_2 - y_1$

As we know the point clipping condition is:

$$x_{wmin} \leq x \leq x_{wmax} \dots\dots\dots(i)$$

$$y_{wmin} \leq y \leq y_{wmax} \dots\dots\dots(ii)$$

## Liang - Barsky algorithm

On putting the value of  $x$  and  $y$  in eqn. (i) & (ii), we get

$$x_{wmin} \leq x_1 + t \Delta x \leq x_{wmax} \dots\dots\dots (iii)$$

$$y_{wmin} \leq y_1 + t \Delta y \leq y_{wmax} \dots\dots\dots (iv)$$

Liang-Barsky express four inequalities with two parameter  $p$  and  $q$  are as follows:

$$t * p_i \leq q_i \quad \text{where, } i=1,2,3,4$$

Where parameter  $p$  and  $q$  are defined as:

$$p_1 = -\Delta x$$

$$q_1 = x_1 - x_{wmin}$$

$$p_2 = \Delta x$$

$$q_2 = x_{wmax} - x_1$$

$$p_3 = -\Delta y$$

$$q_3 = y_1 - y_{wmin}$$

$$p_4 = \Delta y$$

$$q_4 = y_{wmax} - y_1$$



## Liang - Barsky algorithm

Following observations can be :

- ⊕ if  $p_i = 0$  : Line is parallel to the clipping boundary.
- ⊕ if  $q_i < 0$  : Line is completely outside the clipping boundary.
- ⊕ if  $p_i < 0$  : Line proceeds from outside to inside of the clipping boundary.
- ⊕ if  $p_i > 0$  : Line proceeds from inside to outside of the clipping boundary.

i.e ,

**Line intersect the clipping boundary**



## Liang - Barsky algorithm

This algorithm calculate value parameter  $t$ :  $t_1$  and  $t_2$ .

- The value of  $t_1$  is determined checking the edge for which line proceed from outside to inside ( $p_i < 0$ ). **The maximum value is taken.**
  - The value of  $t_2$  is determined checking the edge for which line proceed from inside to outside ( $p_i > 0$ ). **The minimum value is taken**
- if  $t_1 > t_2$ , the line is **completely outside the window** and it can be **rejected**.

Otherwise, the value of  $t_1$  &  $t_2$ , are substituted in the parametric equations to get the intersection points of the clipped line.

# Liang - Barsky algorithm

## Algorithm :

1. Read two endpoints of line, say  $p_1 (x_1, y_1)$  and  $p_2 (x_2, y_2)$  .
2. Read the coordinates of windows ,say  $(x_{wmin}, y_{wmin}, x_{wmax}, y_{wmax})$ .
3. Calculate the value of parameter  $p_i$  and  $q_i$ , for  $i = 1, 2, 3$  & 4.

$$p_1 = -\Delta x$$

$$q_1 = x_1 - x_{wmin}$$

$$p_2 = \Delta x$$

$$q_2 = x_{wmax} - x_1$$

$$p_3 = -\Delta y$$

$$q_3 = y_1 - y_{wmin}$$

$$p_4 = \Delta y$$

$$q_4 = y_{wmax} - y_1$$

## Liang - Barsky algorithm



AMBITION GURU

4. If  $p_i = 0$ , then

{

Line is parallel to the clipping boundary.

Now, if  $q_i < 0$  then

{

Line is completely outside the clipping boundary.

}

}

else

Check for intersection point.

5. Initialize values  $t_1 = 0$  and  $t_2 = 1$ .

6. Calculate values for  $t = \frac{q_i}{p_i}$  for  $i = 1, 2, 3, 4$ .



## Liang - Barsky algorithm

7. Select values of  $\frac{q_i}{p_i}$ , where ( $p_i < 0$ ), assign maximum value out of them as  $t_1$ .

8. Select values of  $\frac{q_i}{p_i}$ , where ( $p_i > 0$ ), assign minimum value out of them as  $t_2$ .

9. If  $t_1 < t_2$ , then calculate intersection points are as :

$$\{ xx_1 = x_1 + t_1 * \Delta x$$

$$yy_1 = y_1 + t_1 * \Delta y$$

$$xx_2 = x_1 + t_2 * \Delta x$$

$$yy_2 = y_1 + t_2 * \Delta y \}$$

Draw clipped line ( $xx_1, yy_1, xx_2, yy_2$ )

10. Stop

## Liang - Barsky algorithm



AMBITION GURU

Q1. Find the clipping coordinates for a line  $z_1 z_2$  where  $z_1=(10,10)$  and  $z_2=(60,30)$ , against window with  $(x_{wmin}, y_{wmin}) = (15,15)$ , and  $(x_{wmax}, y_{wmax}) = (25,25)$ .

**Solution:**

Here, The line coordinates are:

$$\begin{array}{ll} x_1 = 10 & y_1 = 10 \\ x_2 = 60 & y_2 = 30 \end{array}$$

Now, the window coordinates are:

$$\begin{array}{ll} x_{wmin} = 15 & y_{wmin} = 15 \\ x_{wmax} = 25 & y_{wmax} = 25 \end{array}$$

## Liang - Barsky algorithm



1. Calculate the value of parameter  $p_i$  and  $q_i$ , for  $i = 1, 2, 3$  & 4.

$$p_1 = -\Delta x$$

$$q_1 = x_1 - x_{wmin}$$

$$p_1 = -50$$

$$q_1 = -5$$

$$\frac{q_1}{p_1} = 0.1$$

$$p_2 = \Delta x$$

$$q_2 = x_{wmax} - x_1$$

$$p_2 = 50$$

$$q_2 = 15$$

$$\frac{q_2}{p_2} = 0.3$$

$$p_3 = -\Delta y$$

$$q_3 = y_1 - y_{wmin}$$

$$p_3 = -20$$

$$q_3 = -5$$

$$\frac{q_3}{p_3} = 0.25$$

$$p_4 = \Delta y$$

$$q_4 = y_{wmax} - y_1$$

$$p_4 = 20$$

$$q_4 = 15$$

$$\frac{q_4}{p_4} = 0.75$$

➤  $t_1 = \max(0, 0.25, 0.1) = 0.25$ , for the values ( $p_i < 0$ )

➤  $t_2 = \min(0.3, 0.75, 1) = 0.3$ , for the values ( $p_i > 0$ )

## Liang–Barsky algorithm

here,  $t_1 < t_2$ ,

Calculate the intersection points:

$$\begin{aligned}xx_1 &= x_1 + t_1 * \Delta x \\&= 10 + 0.25 * 50 \\&= 22.5\end{aligned}$$

$$\begin{aligned}yy_1 &= y_1 + t_1 * \Delta y \\&= 10 + 0.25 * 20 \\&= 15\end{aligned}$$

$$\begin{aligned}xx_2 &= x_1 + t_2 * \Delta x \\&= 10 + 0.3 * 50 \\&= 25\end{aligned}$$

$$\begin{aligned}yy_2 &= y_1 + t_2 * \Delta y \\&= 10 + 0.3 * 20 \\&= 16\end{aligned}$$

The Clipped Line end  
points are  
( 22.5,15) & (25, 16)



## Questions

1. Prove that two successive rotations are additive.
2. Why is homogeneous coordinates used for transformation computations in computer graphics ? Explain.
3. Explain 2D viewing pipeline in brief.
4. Explain in detail about the Cohen Sutherland line clipping algorithm with an example.
5. Explain about the Sutherland Hodgeman polygon clipping algorithm with an example.
6. Rotate the point (2,-4) about the origin 30 degree in anti clockwise direction.
7. Use Clip a line A(-1,4) and B(3,8) using the Cohen- Sutherland algorithm with window coordinates (-3,1) and (2,6).
8. Use the Cohen - Sutherland algorithm to clip the line P1(70,20) and P2(100,10) against a window lower left-hand corner (50,10) and upper right-hand corner (80,40).
9. Reflect an object (2, 3), (4, 3), (4, 5) about line  $y = x + 1$
10. Define clipping. Explain different types of clipping in brief.
11. Given a triangle with vertices A(2,3), B(5,5), C(4,3) by rotating 90 degrees about the origin and then translating two units in each direction. Use the homogeneous transformation matrix to find the new vertices of the triangle.
12. Why Liang Barsky Line Clipping Algorithm is efficient than Cohen Sutherland Algorithm? Explain the clipping procedure of Liang Barsky algorithm with suitable example.
13. Reflect a line segment having end points (9,3) and (12,10) about a line  $X=7$ . Draw initial and final result graph as well.

THANK YOU  
Any Queries ?